Green Chef Packaging Analysis: Customer Experience, Competitive Landscape, and the Case for Change

Prepared by:

Aakash Nihalaney, Avi Gupta, Harsh Sahu, Om Kapdoskar

Faculty Advisor: Prof. Sean Raines

Sponsor Advisor: Kayla Baber

05/04/2026

PAMPLIN COLLEGE OF BUSINESS VIRGINIA TECH

GREEN CHEF

Table of Contents

1. Executive Summary

Green Chef is a USDA certified organic meal kit brand owned by HelloFresh, serving a customer base that is 75% sustainability conscious and 50% self identified premium shoppers. Despite strong brand equity and a genuine commitment to environmental responsibility, a measurable gap exists between the values Green Chef stands for and the packaging experience its customers receive every week. This project was commissioned to close that gap, drawing on a January 2024 packaging survey of 619 customers, twelve months of Chattermill review data, Q1 2026 cancellation records covering over 21,000 comments, and Green Chef's internal SAP pricing data.

The findings are consistent across all four sources. Plastic reduction is the single most requested packaging improvement at 48% of open ended responses, ice pack disposal is second at 30%, and 4,211 Q1 2026 cancellations contain packaging or freshness related language, 19.2% of all cancellations in that period. At the ingredient level, over 80% of plastic packaging ends up in the garbage not because customers are indifferent but because they lack accessible disposal guidance.

The economic analysis confirms a combined three year net present value of $158,030 across two packaging tiers. Tier 1, the addition of Nutri-Ice at $0.20 per box, delivers a 5.3% ROI and 11.4 month payback. Tier 2, a full sustainable materials transition, delivers 28.4% ongoing ROI with break-even at 25.6 months.

The team delivers three recommendations: a phased packaging overhaul supported by a Weather-Smart AI Packing Assistant integrated with Green Chef's existing Katana ERP and Pick to Light systems; a tiered marketing strategy to convert packaging improvements into measurable retention and reacquisition outcomes; and a QR code powered disposal webapp providing zip code specific recycling guidance for every packaging component. Green Chef has the brand equity, the customer base, and the operational foundation to lead the meal kit industry on sustainable packaging. This report provides the roadmap to do so.

2. Introduction

The meal kit industry has grown significantly over the past decade, with Green Chef establishing itself as the premium, health focused leader through its USDA certified organic ingredients and longstanding commitment to sustainability. As consumer expectations around environmental responsibility continue to rise, a measurable gap has emerged between the brand's values and the packaging experience customers receive every week. This gap is showing up in satisfaction scores, cancellation data, and competitive comparisons with brands that have moved faster on sustainable packaging.

This project was commissioned by Green Chef and HelloFresh to address that gap directly. Green Chef's own January 2024 research found that 48% of customers cited plastic reduction as their top packaging concern, and over half reported that excess plastic negatively impacts their experience. Q1 2026 cancellation data reinforces the scale of the opportunity, with over 21,000 cancellation comments recorded in the first eight weeks of the year, a meaningful portion of which reference packaging dissatisfaction as a contributing factor. Despite Green Chef's broader sustainability commitments, the majority of its ingredient level packaging remains plastic, which customers widely perceive as inconsistent with the brand's premium, sustainable identity.

The purpose of this project is to identify the most impactful and feasible improvements Green Chef can make across its packaging operations and customer experience, grounded in customer data, competitive analysis, and economic evidence. The scope covers every physical component of the meal kit box including the exterior shipping box, ice packs, produce packaging, spice sachets, sauce containers, and individual ingredient wrappers, as well as the warehouse operations and post delivery experience that shape how customers interact with those materials once the box arrives.

Green Chef has a genuine opportunity to turn its packaging from a recurring source of customer frustration into a tangible competitive advantage. With Hungryroot already winning customers on sustainable packaging and Green Chef's own data showing that a transition to eco friendly

materials would positively impact brand perception for over 70% of customers, the case for change is clear. This report presents the findings of that analysis and the path forward.

3. Background

The meal kit delivery industry emerged in the early 2010s as a response to growing consumer demand for convenient, home cooked meals without the burden of grocery planning. What began as a niche subscription concept quickly expanded into a multi billion dollar global market, attracting millions of subscribers across North America, Europe, and beyond. The COVID-19 pandemic served as a significant accelerant for the category, driving a surge in new subscribers as consumers sought safe, reliable alternatives to grocery shopping and restaurant dining. By 2020 the global meal kit market was valued at approximately $10 billion.

As the world reopened the industry entered a more complex phase. Subscriber growth slowed, churn rates remained elevated across the category, and the definition of a winning meal kit brand shifted. Convenience alone was no longer sufficient. Customers began evaluating meal kit services against nutritional quality, ingredient transparency, environmental responsibility, and the overall experience of receiving and unpacking a box every week.

HelloFresh was founded in Berlin, Germany in 2011 and has since grown into the world's largest meal kit company, serving millions of active customers each week across multiple countries. In 2018 HelloFresh acquired Green Chef, recognizing its differentiated positioning in the premium and health focused segment of the market. Rather than absorbing Green Chef into its core brand, HelloFresh preserved its distinct identity, allowing it to continue operating as a standalone premium offering within the broader portfolio. Today Green Chef holds USDA certified organic status, was the first meal kit company to receive CCOF certification, and serves customers across Mediterranean, Plant Based, Keto, and Gluten Free dietary lifestyles. It offsets 100% of its carbon emissions and plastic use, operates on 100% renewable electricity, and has donated over 4.5 million pounds of food to food insecure communities since 2019.

These commitments form the foundation of a brand its customers genuinely believe in. The opportunity this project addresses is one of alignment, ensuring that the physical experience of receiving a Green Chef box every week reflects the same values that drew customers to the brand

in the first place. As consumer sophistication around sustainability continues to grow and competitors raise the bar on eco-friendly packaging and customer experience, Green Chef is well positioned to lead. This project provides the strategy and evidence base to do so.

4. Technical Motivation for Project

Transitioning to sustainable packaging in the meal kit industry is not simply a matter of swapping one material for another. It sits at the intersection of food safety regulation, cold chain logistics, material science, and large-scale operational execution. Meal kit ingredients must remain at safe temperatures throughout a delivery journey spanning one to three days, and plastic has historically been the material of choice because it is moisture resistant, lightweight, durable, and capable of forming an airtight seal. Most sustainable alternatives do not naturally replicate these properties. Paper is vulnerable to moisture, limiting its application for sauces and wet produce. Compostable films frequently require industrial composting infrastructure to break down as intended. Any material change must also satisfy FDA food safety requirements, and at the scale Green Chef and HelloFresh operate, even a targeted swap carries significant implications across supplier relationships, distribution center operations, and packaging line equipment.

The cold chain challenge is further complicated by variation in weather conditions across Green Chef's delivery geography. Without a dynamic system for adjusting packaging based on destination conditions, a box shipped to Arizona in August and one shipped to Minnesota in January may leave the warehouse identically packed despite facing entirely different thermal environments. Green Chef's existing Katana ERP and Pick to Light warehouse systems provide a strong operational foundation, and any technical solution should integrate with rather than replace them. The opportunity is to layer intelligence on top of existing infrastructure, using real time data to make smarter packing decisions at the order level without disrupting the workflows warehouse teams already rely on.

Beyond the physical packaging there is a technical gap in the post delivery experience. Disposal infrastructure varies significantly by location, and without accessible, location specific guidance, even the most well intentioned customers default to the garbage. Addressing this gap does not require changes to physical materials but does require a digital solution simple enough to use in

the moment a customer is standing at their recycling bin. Taken together these considerations point toward a packaging strategy that is phased, operationally integrated, and supported by digital tools that extend the customer experience beyond the moment of unboxing.

5. Economic Motivation for Project

Customer retention is one of the most significant economic challenges facing the meal kit industry, and Green Chef's Q1 2026 data makes the scale of that challenge concrete. With approximately 78,000 active customers and weekly cancellation rates ranging from 3.9% to 6.6% across the first seven weeks of the year, Green Chef recorded over 21,000 cancellation comments in that period alone. Analysis of those comments shows that 4,211 contain language related to packaging, freshness, or ingredient quality concerns. These are not cancellations driven by price sensitivity or personal circumstances beyond Green Chef's control. They reflect a specific and addressable gap between the experience customers expected and the one they received, and the financial return on closing that gap is both calculable and realistically achievable.

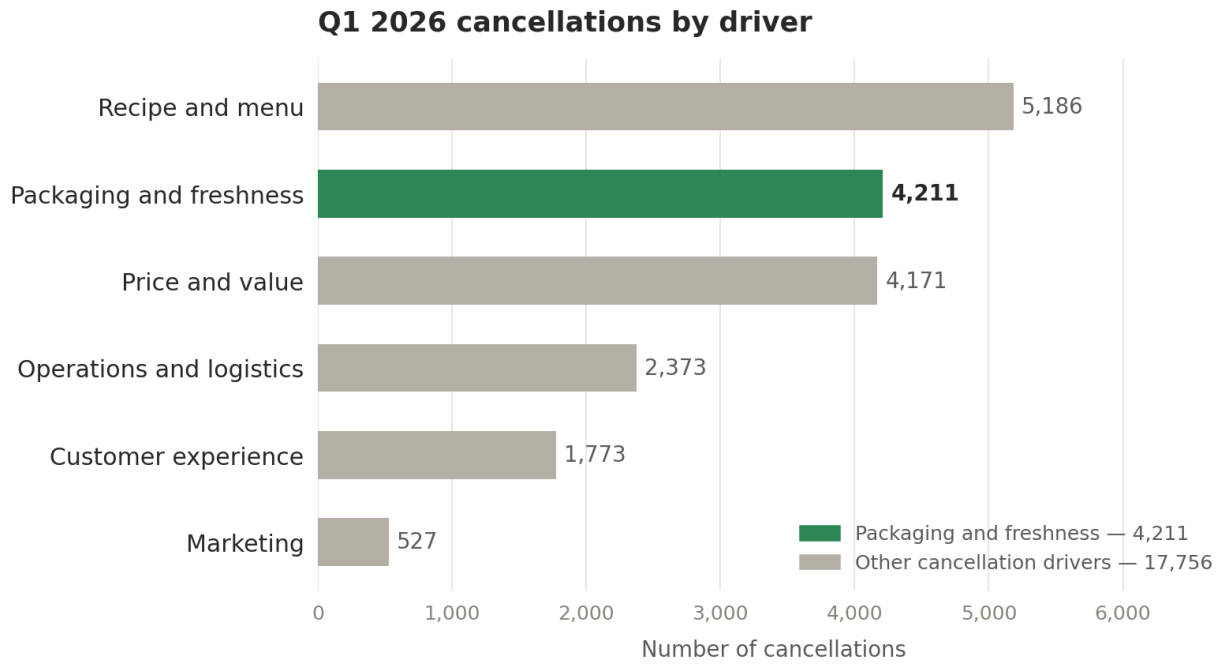


Figure 5.1.

The January 2024 packaging research adds a forward looking dimension to this picture. Among active customers, 59% said a switch to eco-friendly paper packaging would positively impact their likelihood to order another box, and 53% of recently cancelled customers said the same. Green Chef's own research also found that 71% of active customers said a switch to paper packaging would positively impact their overall perception of the brand and 67% said it would improve their satisfaction with the service. Sustainable packaging investment therefore functions not only as a retention tool for current subscribers but as a reacquisition lever for lapsed ones, a dual return that significantly strengthens the economic case for action. Hungryroot, Green Chef's most significant competitive threat, is already using packaging innovation as a customer acquisition tool, and Green Chef's own subscribers have cited Hungryroot's garden safe ice packs as the benchmark for what they want to see. The economic cost of that dynamic compounds over time.

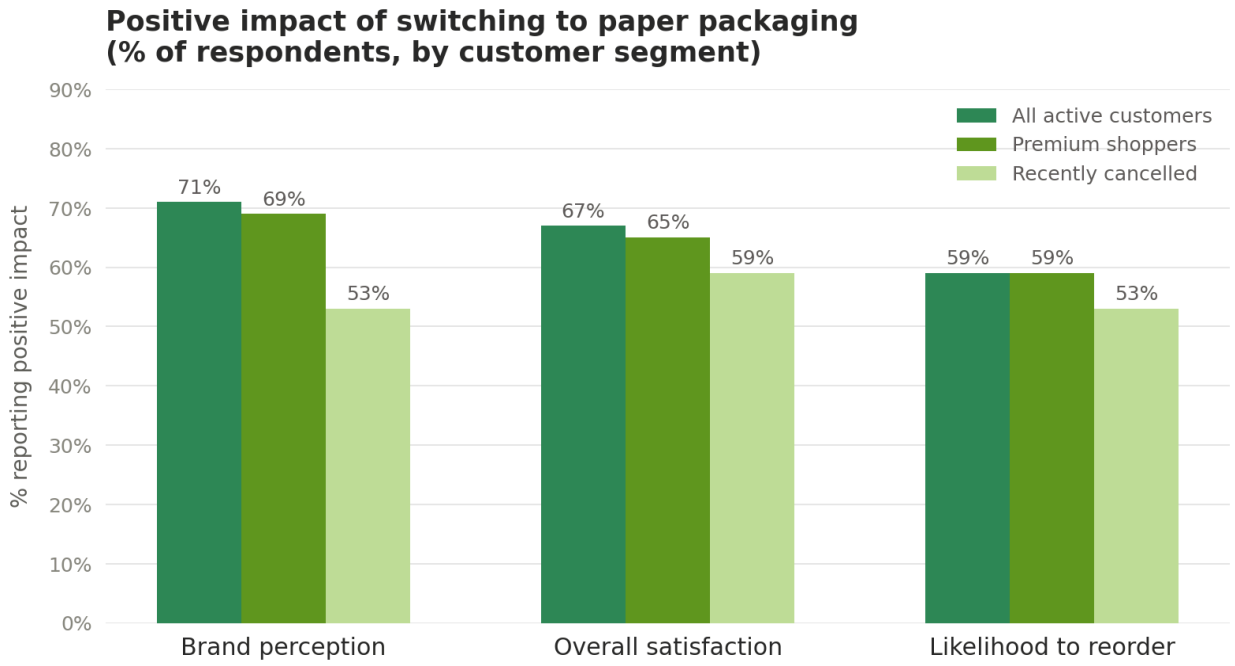


Figure 5.2.

Sustainable packaging does carry real incremental cost, and the January 2024 research is clear that less than half of customers would be willing to pay more as a result of a materials switch. This makes it important to frame the return on investment through the lens of lifetime customer value rather than short term margin. The cost of retaining a customer for an additional several

months through improved packaging far exceeds the incremental material cost of making that change. The economic analysis in Section VIII quantifies this tradeoff in full using net present value, payback period, and return on investment modeling. The purpose of this section is to establish the foundational logic: the financial case for investing in sustainable packaging is strong, the customer data supports it, and the cost of continued inaction is measurable and growing.

6. Problem Statement

Green Chef has built its reputation as the most sustainable meal kit brand in the United States, yet the packaging customers receive every week tells a different story. Despite offsetting 100% of its carbon emissions and plastic use, the majority of its ingredient level packaging remains plastic, and Green Chef's own research shows that 86% of plastic spice packaging, 81% of plastic meat packaging, and 77% of plastic produce packaging ends up in the garbage. For a customer base that is 75% sustainability conscious and 50% self identified premium shoppers, this disconnect is not abstract, it is visible every time they unpack a box.

The consequences are measurable. Q1 2026 data shows over 21,000 cancellation comments in the first eight weeks of the year, with 4,211 containing packaging or freshness related language. A further 158 are formally coded as Ecological Issues. At the same time, Green Chef's current warehouse operations apply identical packing configurations regardless of the weather conditions at each delivery destination, creating avoidable freshness and spoilage risk in transit. And once the box arrives, customers who want to dispose of materials responsibly have no component level guidance to help them do so.

These are not structural problems inherent to the meal kit model. They are specific, addressable gaps between what the brand promises and what customers experience, and the data makes clear that closing them would have a direct and positive impact on retention, satisfaction, and brand perception.

6.1. Statement of Work

The purpose of this project is to develop a comprehensive and actionable packaging strategy for Green Chef that addresses the disconnect between its sustainability brand identity and the current customer experience. The Virginia Tech student team is responsible for conducting a thorough analysis of Green Chef's existing packaging across all components of the meal kit box, identifying the most impactful opportunities for improvement, and delivering a final set of recommendations supported by customer data, competitive benchmarking, and economic analysis.

The scope of this project covers every packaging component within the Green Chef meal kit box including the exterior shipping box, ice packs, produce packaging, spice sachets, sauce containers, meat packaging, and recipe cards. It also extends to the warehouse operations that govern how those components are selected and packed, and the post delivery experience customers have when disposing of materials. Each component and system has been evaluated against three criteria: environmental impact, customer experience, and operational and economic feasibility.

The final deliverable is this written report and an accompanying presentation, providing Green Chef with a clear and prioritized roadmap for transitioning to more sustainable packaging as part of its 2026 product roadmap.

6.1.1. Glossary of Terms and Acronyms

Green Chef - A premium, USDA certified organic meal kit brand owned and operated by HelloFresh, serving the United States market.

HelloFresh - The parent company of Green Chef and the world's largest meal kit delivery service, headquartered in Berlin, Germany.

Meal Kit - A subscription based food delivery service that provides pre portioned, fresh ingredients along with step by step recipe instructions for home cooking.

Cold Chain - The temperature controlled supply chain required to keep perishable food products fresh and safe from the point of production through to the customer's door.

Churn Rate - The percentage of subscribers who cancel their subscription within a given time period.

Sustainable Packaging - Packaging designed to minimize environmental impact through the use of recyclable, compostable, biodegradable, or reusable materials.

USDA Organic - A certification issued by the United States Department of Agriculture indicating that a product has been produced using approved organic methods.

CCOF - California Certified Organic Farmers, an independent organic certification organization. Green Chef is the first meal kit company to receive CCOF certification.

Chattermill - A customer feedback analytics platform used by Green Chef to collect and analyze qualitative and quantitative customer review data.

CX - Customer Experience. The overall perception a customer has of a brand based on all interactions throughout their relationship with the company.

ClimaCell - A plant based insulation material currently used by Green Chef in its shipping boxes that is curbside recyclable in most regions of the United States.

Nutri-Ice - A food safe, non toxic gel cooling insert used to extend cold chain integrity during last mile delivery without requiring changes to existing packaging infrastructure.

Katana ERP - The enterprise resource planning system used by Green Chef to manage orders and inventory across its warehouse operations.

Pick to Light - A warehouse guidance system that uses bin mounted lights to direct workers to the correct items and quantities during the packing process.

Compostable Packaging - Packaging that breaks down into natural materials in a composting environment within a specified timeframe, leaving no toxic residue.

6.1.2. Project Goals

The primary goal of this project is to provide Green Chef with a data driven, actionable packaging strategy that is realistic, phased, and aligned with its premium brand positioning. The following goals guide the work.

Goal 1: Identify and prioritize packaging pain points. Using Green Chef's customer feedback, cancellation and pause data, and January 2024 packaging research, identify which packaging components are causing the greatest customer dissatisfaction and contributing most to churn.

Goal 2: Evaluate sustainable packaging alternatives. Research and assess viable alternatives for each packaging component against criteria including environmental impact, food safety compliance, cold chain performance, cost, and customer perception.

Goal 3: Benchmark against competitors. Analyze how direct and indirect competitors including Hungryroot, Sunbasket, and Daily Harvest are approaching sustainable packaging to identify gaps and opportunities for Green Chef to differentiate.

Goal 4: Develop an economic analysis. Quantify the financial impact of the current packaging gap and model the return on investment of proposed changes including payback period and net present value.

Goal 5: Deliver actionable recommendations. Present a prioritized set of recommendations that Green Chef can realistically implement as part of its 2026 product roadmap, spanning physical packaging materials, warehouse operations, and the post delivery customer experience.

6.1.3. Administrative Procedures

This project is conducted as part of the Virginia Tech Masters of Science in Business Analytics capstone program under the supervision of Faculty Advisor Professor Sean Raines. The sponsor advisor is Kayla Baber, Product Manager at Green Chef and HelloFresh.

Communication. Primary communication between the student team and the sponsor advisor takes place via email and scheduled online meetings. The sponsor advisor is available for one hour each week during office hours to answer questions, provide feedback, and validate the team's direction. All formal deliverables are submitted to both the faculty advisor and sponsor advisor for review.

Confidentiality. All data, research, and proprietary information provided by Green Chef and HelloFresh is treated as confidential in accordance with the non-disclosure agreement signed by each team member at the outset of the project. This information is used solely for the purposes of this capstone project and is not shared outside the project team.

6.1.4. Timeline

This project ran over approximately six weeks from late March through early May 2026, structured across four phases with formal deliverables at each stage.

Phase 1: Customer Discovery and Need State Analysis - March 20 to April 1. The team reviewed all customer feedback data provided by Green Chef, including the January 2024 packaging research, Chattermill review data, and Q1 2026 cancellation and pause records. Key pain points and need states were identified and categorized. The first formal deliverable, the Mid Project Review presentation and accompanying presentation materials, was submitted on April 1.

Phase 2: Competitor Landscape and Initial Analysis - April 1 to April 13. The team conducted competitive benchmarking across Hungryroot, Sunbasket, and Daily Harvest and completed the initial analysis of Green Chef's current packaging across all box components. The Executive Summary and Report draft was submitted on April 13, receiving a score of 92 out of 100.

Phase 3: Concept Development and Economic Analysis - April 14 to April 28. The team developed and evaluated product concepts, selected and refined the three final recommendations, and completed the economic analysis including net present value, payback period, and return on investment modeling. Initial feedback from team members was submitted on April 17.

Phase 4: Final Deliverables - April 28 to May 6. The team compiled all findings into this written report, the final presentation, and accompanying presentation materials. The Capstone Problem Statement, Project Focus, and Recommendations document was submitted on May 3. All remaining final deliverables including the Executive Summary and Report, final presentation, pres

6.2. Deliverables

This project produced the following formal deliverables over the course of the engagement.

Mid Project Review Presentation - April 1. A presentation covering the initial customer discovery findings, identified pain points and need states, and early competitive benchmarking observations. Accompanied by a separate presentation materials submission.

Executive Summary and Report Draft - April 13. A draft of this report covering the initial analysis, basis for the analysis, metrics used, and status of Green Chef's current packaging design and market position. Reviewed by both the faculty and sponsor advisors prior to final revision.

Capstone Problem Statement, Project Focus, and Recommendations - May 3. A summary document capturing the refined problem statement, confirmed project focus, and the three final recommendations developed by the team.

Final Presentation and Presentation Materials - May 6. A professional presentation of the complete project findings, economic analysis, and recommendations, prepared for Green Chef leadership and the Virginia Tech capstone program.

Final Executive Summary and Report - May 6. This document, representing the complete written record of the project including all analysis, findings, economic modeling, and recommendations.

Team Member Evaluations - May 6. Individual evaluations completed by each team member as part of the Virginia Tech capstone program requirements.

7. Body

7.1. Initial Analysis

The following analysis examines Green Chef's current packaging across three dimensions: the voice of the customer, the competitive landscape, and the broader market environment. The goal is to establish a clear, evidence based understanding of where the packaging experience is falling short, how significant the opportunity is, and what addressing it effectively would mean for the business.

The analysis draws on four primary data sources: Green Chef's January 2024 packaging research survey, qualitative and quantitative review data collected through Chattermill over the last twelve months, internal cancellation and pause data from Q1 2026, and a formal packaging research presentation prepared by Green Chef's internal team. By triangulating across all four sources, the analysis moves beyond any single data point to build a complete and consistent picture of the customer experience.

What emerges is not a story of isolated complaints but a pattern of cross validated dissatisfaction centered on a specific and addressable issue. Green Chef's packaging is functioning as a detractor in an experience that is otherwise strongly positive, customers consistently celebrate the quality of the food, the boldness of the flavors, and the cooking experience itself. The opportunity is to bring the packaging in line with the rest of what makes Green Chef worth subscribing to.

7.1.1. Basis for the Analysis

The analysis presented in this report is grounded in customer data provided directly by Green Chef and HelloFresh. Four primary sources inform the findings.

The first is a quantitative packaging survey conducted by Green Chef in January 2024, distributed to 25,000 customer records and completed by 619 respondents, of whom 594 were active subscribers and 25 were recently cancelled customers. The second is

qualitative and quantitative customer review data collected through Chattermill over the last twelve months, comprising 10,000 positive and 10,000 negative reviews tagged by theme, ingredient, and customer loyalty level. The third is internal cancellation and pause data from Q1 2026, covering 21,967 cancellation comments and 4,927 pause comments across the first eight weeks of the year. The fourth is a formal packaging research presentation prepared by Green Chef's internal team summarizing key customer insights around packaging perception, disposal behavior, and sustainability preferences.

Together these sources provide a multi dimensional view of the customer experience that moves beyond sentiment alone to connect packaging dissatisfaction directly to measurable business outcomes.

7.1.2. Metrics Used for the Analysis

The analysis relies on a combination of quantitative and qualitative metrics drawn across all four data sources, selected for their relevance to two core questions: how significant is the packaging opportunity, and what is the business case for addressing it.

Churn and retention metrics form the foundation. Weekly cancellation and pause rates, total active customer counts, and cancellation reason categories establish the scale of the problem and allow packaging related dissatisfaction to be isolated and quantified within the broader pool of cancellation drivers.

Customer sentiment and feedback metrics add depth. Review sentiment scores, theme frequency across positive and negative reviews, and open ended cancellation comments reveal the consistency and intensity of packaging related frustration across thousands of independent customer voices.

Disposal and engagement metrics capture how customers actually interact with packaging in practice. Disposal rates by material type, packaging engagement levels, and importance and frequency ratings for specific packaging attributes reveal not just what customers say they want but how they behave when no one is asking.

Finally, forward looking metrics connect customer sentiment to financial outcome. Customers reported likelihood to reorder, likelihood to recommend, and willingness to pay following a switch to sustainable packaging form the bridge between the customer experience findings and the economic analysis that follows in Section VIII.

7.1.3. Status of Current Design / Process / Product / Market

Current Packaging

Green Chef's meal kit box contains several distinct packaging components, each serving a specific functional role in the cold chain delivery process. The exterior cardboard shipping box and paper kit bag form the outer layer of the delivery. Individual ingredient packaging includes plastic produce bags, plastic spice sachets, plastic sauce containers, plastic meat packaging, and ice packs. A paper recipe card is included in every box. The box lining currently uses ClimaCell, a plant based insulation material that is curbside recyclable in most regions, and ice packs are supplied by Pelton Shepherd.

Customer perception of these components varies significantly by material. The paper based elements including the cardboard shipping box, paper kit bag, and recipe card are widely perceived as sustainable and receive positive sentiment. The plastic components tell a different story. Disposal behavior reflects this clearly: while 84% of customers recycle the exterior cardboard box, 86% of plastic spice packaging, 81% of plastic meat packaging, and 77% of plastic produce packaging ends up in the garbage. The ice pack presents its own challenge, with 54% of customers discarding it in the garbage and many citing difficulty with gel disposal and concern about leakage during transit.

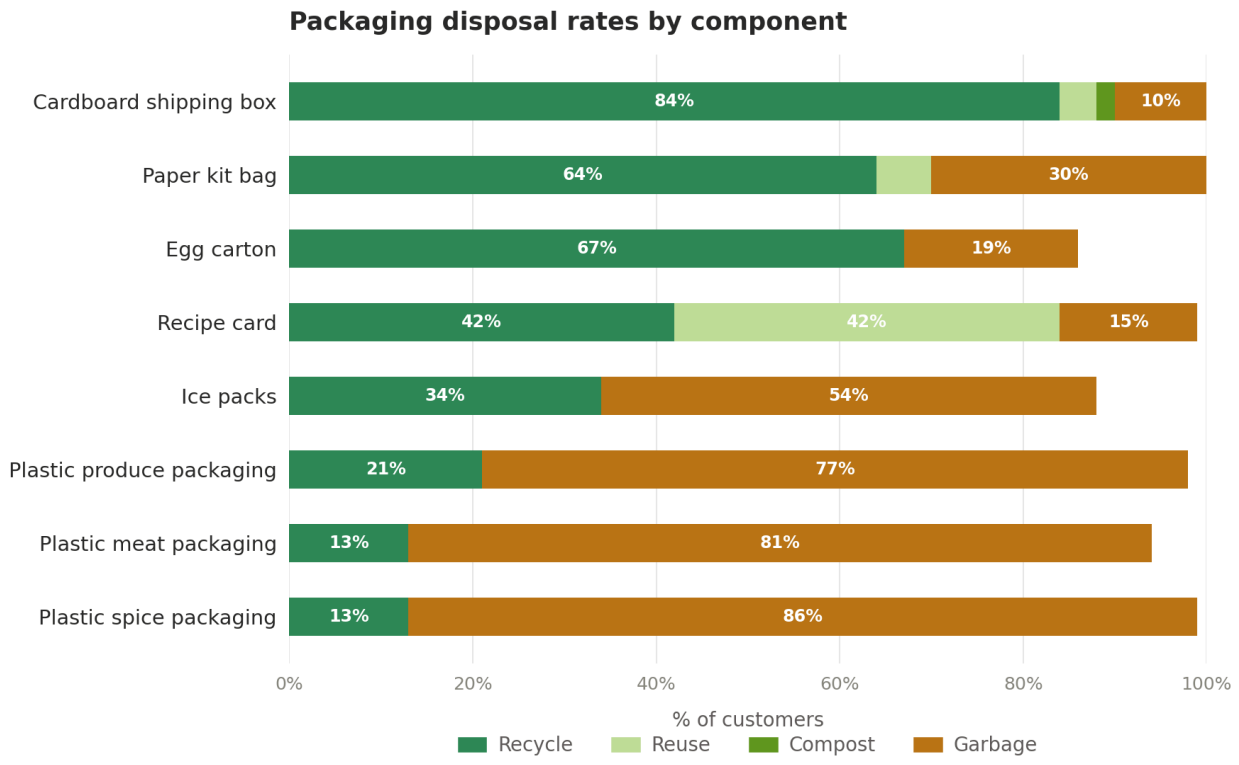


Figure 7.1.

Customer Pain Points

Customer feedback across all four data sources tells a consistent story about where the packaging experience is falling short. Plastic reduction is the single most cited improvement across open ended responses, mentioned by 48% of customers in the January 2024 survey. Ice pack disposal is the second most common concern, cited by 30% of customers. Produce packaging is third at 26%, with customers frustrated by the volume of single use plastic around fresh vegetables and herbs.

The Chattermill data reinforces these findings at scale. Negative reviews frequently reference damaged ingredient packaging, excess plastic, and difficulty recycling. Positive reviews, by contrast, celebrate ingredient quality, bold flavors, and the cooking experience. The contrast is telling: packaging is functioning as a detractor in an experience that is otherwise strongly positive.

Beyond environmental concerns, premium perception is also at stake. Half of Green Chef's customers identify as premium shoppers, and Green Chef's own research found that premium shoppers nearly universally agree that plastic does not feel premium. For a brand charging $12 to $13.50 per serving, that perception gap has direct implications for satisfaction and retention.

Market and Competitive Landscape

The meal kit industry is at an inflection point on sustainability. Hungryroot, identified by Green Chef as its most significant competitive threat, has built strong brand equity on personalization and sustainable packaging. Its ice packs, designed to be drained directly into the garden as plant food, are already being cited by Green Chef's own subscribers as the benchmark for the kind of innovation they want to see. Sunbasket has invested in recyclable and compostable materials as a core brand differentiator. Daily Harvest, operating through frozen single serve products, uses minimal packaging by design and has built strong equity among the health conscious, eco aware consumers that overlap significantly with Green Chef's core audience.

The broader market trend reinforces the urgency. Customers are becoming more sophisticated in evaluating sustainability claims, and brands that cannot demonstrate tangible, material progress on packaging will find it increasingly difficult to retain the loyalty of the consumers who matter most to them. Green Chef's current position in this landscape is one of unrealized potential; the brand has the values, the customer base, the organic credentials, and the organizational backing of HelloFresh to lead the industry on sustainable packaging.

7.1.4. Conclusions of the Initial Analysis

The initial analysis points to a clear and consistent conclusion: packaging is one of the most significant and addressable drivers of customer dissatisfaction at Green Chef, and the opportunity to improve it is both measurable and well supported by the data.

Across all four sources, the same themes emerge with remarkable consistency. Customers want less plastic. They want packaging that is easy to dispose of responsibly. They want

the physical experience of receiving a Green Chef box to feel as premium and values aligned as the brand they have chosen. These are not fringe concerns raised by a vocal minority but majority sentiments expressed across thousands of independent customer voices.

The business impact is already visible in the churn data, and the reacquisition opportunity among lapsed customers is equally significant. Over half of recently cancelled customers said a switch to eco-friendly packaging would positively impact their likelihood to return, which means packaging improvement is both a retention tool and a win back lever.

The competitive analysis adds urgency. Green Chef is operating in a market where sustainability is becoming a baseline expectation rather than a differentiator, and competitors are actively using packaging innovation to attract the customers Green Chef is best positioned to serve. The brand has everything it needs to lead on this issue. The data makes clear that doing so would strengthen its competitive position at a moment when that position most needs defending.

7.1.5. Strategy for Economic Analysis

The economic analysis presented in this section answers a single fundamental question: does the business case for investing in sustainable packaging justify the cost? It approaches the problem from two directions simultaneously; quantifying the revenue being lost to packaging related churn using Green Chef's active customer count, weekly cancellation rates, and the proportion of cancellations attributable to packaging dissatisfaction, and modeling the financial return of addressing it through net present value, payback period, and return on investment.

The analysis assumes that the cost of sustainable packaging innovation must be absorbed rather than passed on to the consumer, consistent with the January 2024 research finding that less than half of customers would be willing to pay more as a result of a materials switch. This makes lifetime customer value central to the model, as the return is realized entirely through improved retention and reacquisition rather than price increases.

All cost inputs are drawn directly from Green Chef's internal SAP pricing data. Churn and revenue inputs are drawn from Q1 2026 cancellation records. Two inputs in the Tier 2 model, the sustainable packaging material premium and the one-time implementation cost, are based on industry benchmarks and should be updated once supplier quotes are confirmed.

8. Economic Analysis

The economic analysis presented in this section quantifies the financial case for Recommendation 1, the phased sustainable packaging overhaul, across both Tier 1 and Tier 2. All cost inputs are drawn directly from Green Chef's internal SAP pricing data. Churn and revenue inputs are drawn from Q1 2026 cancellation records. Two inputs in the Tier 2 model: the sustainable packaging material premium and the one-time implementation cost, are based on industry benchmarks and should be updated once supplier quotes are confirmed.

The model uses a shared set of inputs across both tiers. Green Chef's active customer base of 78,000 subscribers generates approximately $78 in weekly revenue per customer, based on six servings at $13.00 per serving. Retained customers are conservatively assumed to extend their subscription by eight weeks, producing a lifetime value of $624 per retained customer. A 10% discount rate is applied for all net present value calculations. Q1 2026 data recorded 4,211 cancellations containing packaging or freshness related language across the first eight weeks of the year, representing 19.2% of all cancellations in that period.

8.1. Net Cash Flow Diagram

The net cash flow diagrams below illustrate the annual cost, annual benefit, and net position for each tier across a three year horizon.

For Tier 1, the cash flow profile is consistent across all three years. The annual incremental cost of $811,200 and the annual revenue benefit of $853,991 are both present from Year 1, producing a net benefit of $42,791 in each period. There are no one-time costs and no ramp-up phase. The investment generates positive net cash flow from the moment of deployment.

For Tier 2, Year 1 carries the one-time implementation cost of $200,000 alongside the annual incremental cost of $330,452, producing a total outlay of $530,452 against a benefit of $424,320 and a net loss of $106,132. From Year 2 onwards the one-time cost is fully absorbed and the model produces a consistent net benefit of $93,868 annually.

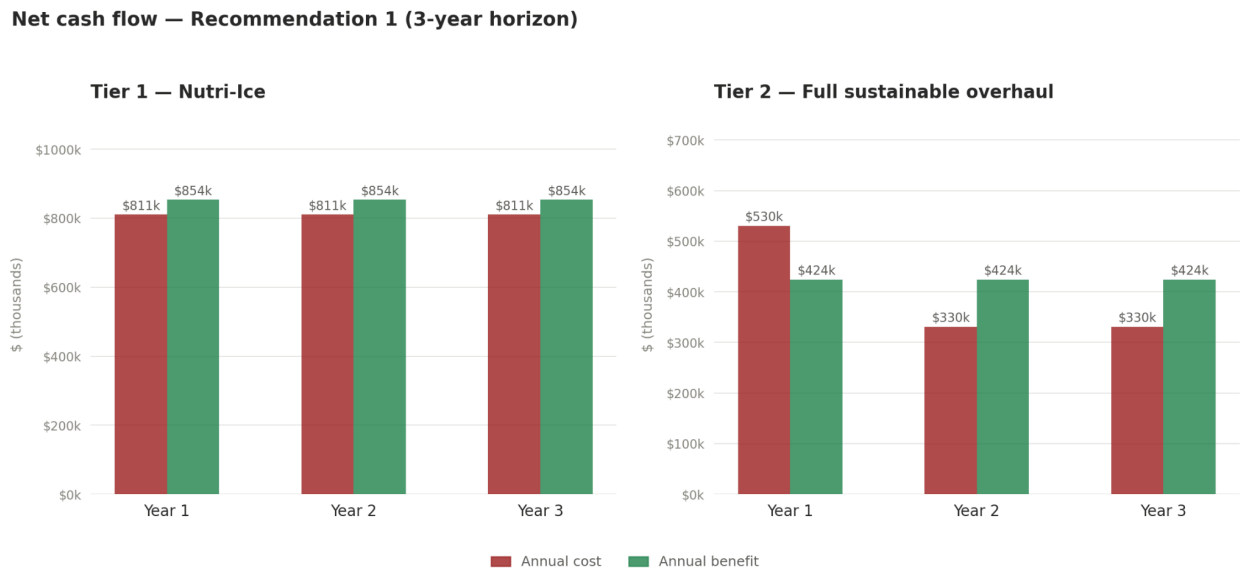


Figure 8.1.

8.2. Payback Period

The payback period measures how long it takes for the cumulative revenue benefit of each tier to recover its total cost outlay.

For Tier 1, the annual incremental cost of $811,200 is recovered by the annual revenue benefit of $853,991 within 11.4 months. The investment pays for itself before the end of its first full year and generates positive returns in every subsequent period.

For Tier 2, the payback calculation includes both the annual incremental cost of $330,452 and the one-time implementation cost of $200,000. The net monthly benefit after ongoing costs is $7,822, against which the $200,000 one-time cost is recovered in 25.6 months. Full cumulative break-even is reached at 2.1 years, partway through Year 3 of the model. Year 1 produces a net loss of $106,132 due to the one-time cost, after which the investment becomes consistently cash flow.

positive.

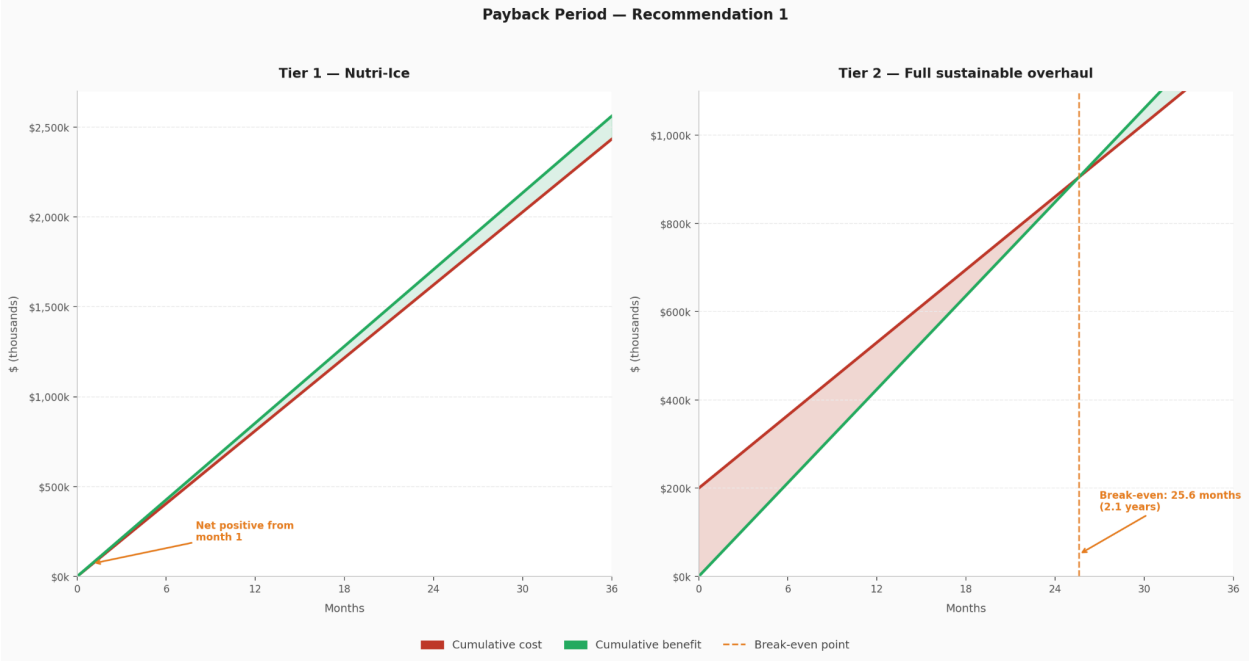


Figure 8.2.

8.3. Net Present Value

Discounting all cash flows at 10% annually, Tier 1 produces a three year net present value of $106,414, with positive discounted returns in each of the three years. Tier 2 produces a three year net present value of $51,616, reflecting the Year 1 net loss offset by the positive discounted returns in Years 2 and 3. Combined, the two tiers generate a three year net present value of $158,030.

The NPV profile of Tier 2 is consistent with a phased capital investment where returns build as the one-time implementation cost is absorbed. The positive combined NPV confirms that the investment case for Recommendation 1 as a whole is financially sound within the three year window, and that returns continue to accumulate beyond it.

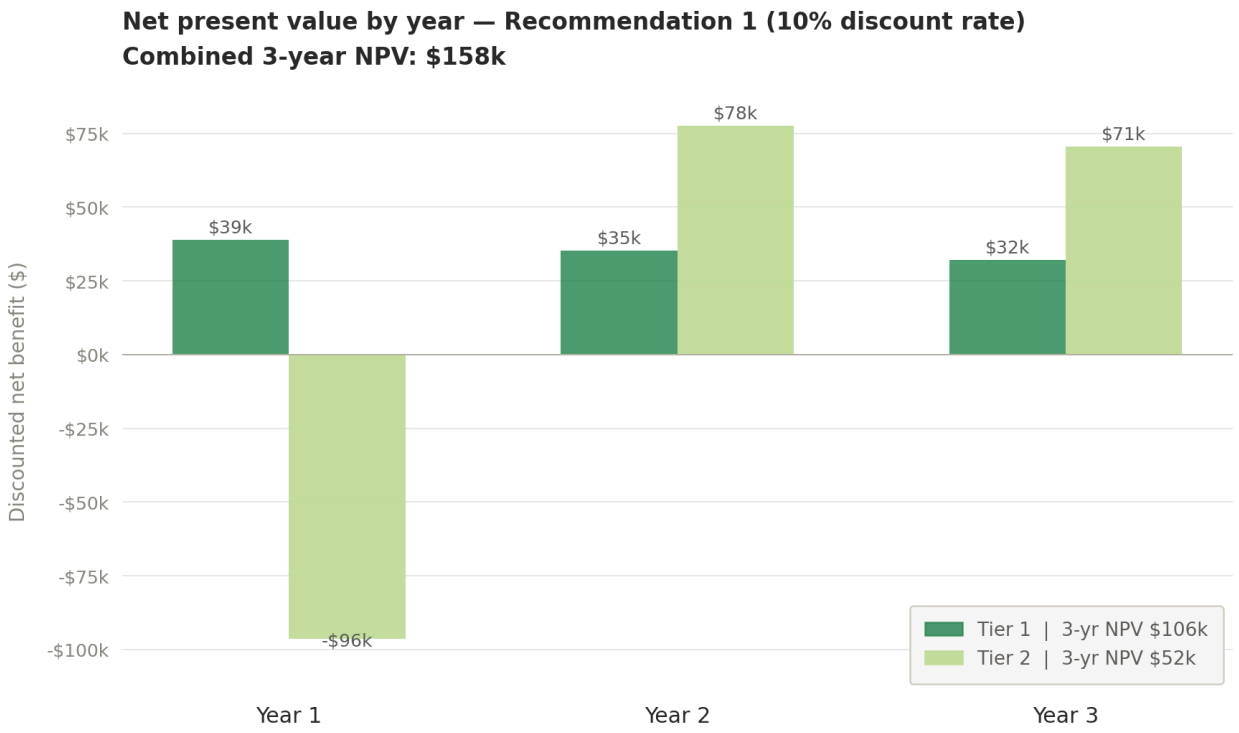


Figure 8.3.

8.4. Return on Investment

Tier 1 delivers a return on investment of 5.3% with an 11.4 month payback. This figure reflects a deliberately conservative set of assumptions: 50% of packaging cancellations are attributed to freshness concerns, 10% of those customers are retained by the Nutri-Ice upgrade, and retained customers extend their subscription by two months on average.

Tier 2 delivers an ongoing return on investment of 28.4%, calculated on annual incremental costs excluding the one-time implementation expense. The benefit draws on three streams: eco churn retention generating $324,480 annually, win-back of lapsed eco-concerned customers generating $37,440, and a brand perception lift conservatively estimated at $62,400, representing just 100 additional customers extending their subscription as a result of the packaging change.

Taken together, Recommendation 1 presents a financially grounded case for phased investment. Tier 1 is the lower risk entry point with a sub-year payback and no capital requirement. Tier 2 is the longer term commitment whose returns build steadily from Year 2 onwards and whose

strategic value in brand equity, competitive positioning, and customer trust extends well beyond what the financial model alone captures.

9. Conclusion

This project set out to identify the most impactful and feasible improvements Green Chef can make to its packaging and customer experience, grounded in the data its own customers have provided. What the analysis reveals is not a brand with a packaging problem so much as a brand with an alignment opportunity; one where the values it stands for, the customers it attracts, and the physical experience it delivers have not yet fully converged.

The data is consistent across every source. Customers want less plastic. They want packaging that is easy to dispose of responsibly. They want the experience of receiving a Green Chef box to feel as premium and intentional as the food inside it. These are majority sentiments expressed across thousands of independent customer voices, and they are showing up in the churn data in ways that are measurable and addressable.

The three recommendations delivered by this project are designed to close that gap in a way that is phased, operationally realistic, and economically grounded. Tier 1 of Recommendation 1 is the most immediate opportunity; a targeted, low risk upgrade to cold chain performance that generates positive returns within its first year and directly addresses the largest addressable segment of packaging driven churn. Tier 2 brings Green Chef's ingredient level packaging in line with its sustainability positioning and builds a durable competitive advantage over a category that is moving quickly in this direction. Recommendation 2 converts those operational improvements into measurable retention and reacquisition outcomes. Recommendation 3 extends the packaging experience beyond the box itself, turning a consistent source of customer frustration into a moment of brand reinforcement.

Taken together these three recommendations represent the closing of a gap that the brand's own customers have clearly and consistently asked it to close. The financial analysis confirms that the investment is sound, the customer research confirms that the appetite is there, and the competitive landscape confirms that the time to act is now. Green Chef has the brand equity, the

customer base, and the organizational backing of HelloFresh to lead the meal kit industry on sustainable packaging. This report provides the evidence base and the roadmap to do so.

10. Recommendations

10.1. Recommendation 1: A Phased Sustainable Packaging Overhaul, Powered by a Weather-Smart AI Packing System

Green Chef's packaging gap spans three dimensions: ingredient freshness during transit, the environmental footprint of plastic ingredient packaging, and the operational systems that govern how boxes are packed. This recommendation addresses all three through a phased approach supported by intelligent warehouse technology.

Tier 1 - Nutri-Ice

Replacing the current Pelton Shepherd ice pack with Nutri-Ice at an incremental cost of $0.20 per box directly targets the 31.3% of negative reviews referencing ingredient spoilage and the 2,106 freshness-driven cancellations recorded every eight weeks. No infrastructure changes are required. The economic case is clean: 5.3% ROI, 11.4 month payback, and a three year NPV of $106,414.

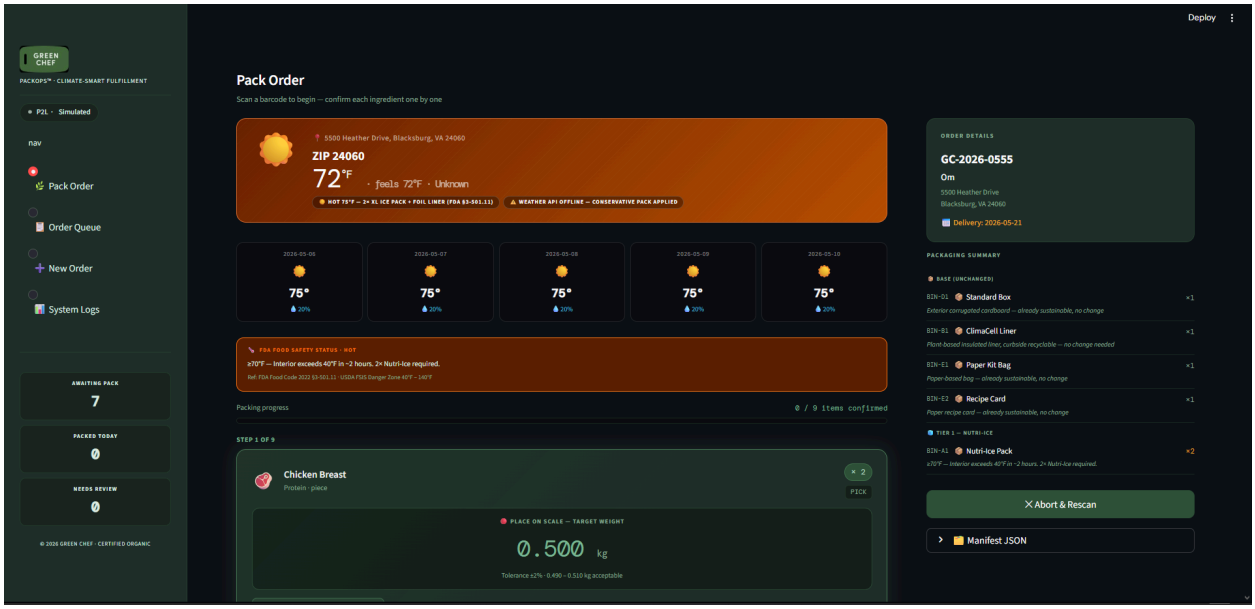
Tier 2 - Full Sustainable Packaging Overhaul

Tier 2 replaces the three plastic autobagger variants currently used at the ingredient level with paper or compostable alternatives, directly responding to the 48% of customers who cited plastic reduction as their top packaging request. At a 15% cost premium over current plastic pricing, the annual incremental cost is $330,452, with a one-time implementation cost of $200,000 covering supplier onboarding and sustainability certification. Total annual benefit across eco retention, win-back, and brand perception lift is $424,320, producing a 28.4% ongoing ROI, a 25.6 month payback, and a three year NPV of $51,616.

The Weather-Smart AI Packing Assistant

Green Chef currently applies identical packing configurations regardless of delivery destination, creating avoidable cold-chain risk across its distribution geography. The

Weather-Smart AI Packing Assistant resolves this by integrating directly with Green Chef's existing Katana ERP and Pick-to-Light systems. It reads live orders, fetches real-time weather data for each destination, and automatically adjusts the packing manifest, upgrading ice packs and adding thermal liners in heat, removing ice packs and adding insulation in cold, and adding waterproofing in precipitation. It signals Pick-to-Light hardware to guide workers to the correct items, displays a color-coded checklist on a touchscreen, and verifies box weight against the manifest before release. If the weather API is unreachable it falls back to regional defaults without interrupting operations. The result is that every box ships with the right configuration for its destination, every time.



10.2. Recommendation 2: A Tiered Marketing Strategy to Convert Packaging Improvements into Measurable Churn Reduction

Operational packaging improvements only reduce churn if customers know about them and experience them as meaningful. This recommendation translates the Tier 1 and Tier 2 changes into retention, win-back, and brand equity outcomes through four coordinated pillars.

The first is the Freshness Promise, launched in parallel with Tier 1. Messaging leads with the customer outcome rather than the feature, ingredients arrive colder and in better condition, every

delivery. Customers who previously submitted negative reviews or cancellation comments citing spoilage should be proactively contacted with a targeted email acknowledging the issue and introducing the improvement, paired with a service guarantee.

The second is the Sustainability Story, built around Tier 2. Rather than positioning the materials transition as a box swap, Green Chef should treat it as a brand moment; publishing a sustainability transparency report, pursuing How2Recycle or equivalent certification, and leading with the change across welcome kits and renewal emails. The goal is to make the improvement visible and verifiable rather than communicating it only through brand copy.

The third is a targeted win-back program. The cancellation data provides an unusually precise foundation for reacquisition because customers who left citing freshness or plastic concerns stated their reason explicitly. Upon Tier 1 launch a win-back sequence should reach former freshness churners directly. Upon Tier 2 launch a parallel sequence should reach eco-concerned churners. Both should demonstrate the specific change rather than assert it.

The fourth is the in-box unboxing moment. A small card explaining the Nutri-Ice insert should accompany Tier 1 deliveries. For Tier 2 the standard packing notes should be replaced with a storytelling card about the new materials and how to dispose of them. Both give customers something tangible to engage with and, for those inclined, something to share on social media without incremental paid spend.

10.3. Recommendation 3: A Smart Packaging Disposal Guide Delivered via QR Code Webapp

Despite 71% of Green Chef's customers having read the recycling information on the exterior shipping box, disposal guidance does not exist at the component level. The result is that customers who want to act on their sustainability values cannot; 86% of plastic spice packaging, 81% of plastic meat packaging, and 77% of plastic produce packaging ends up in the garbage not because customers are indifferent but because the path of least resistance is the only path available to them.

This recommendation addresses that gap through a lightweight mobile-first webapp accessible by scanning a QR code printed directly on each packaging component. A customer scanning the code on their produce bag receives zip-code-specific disposal instructions for that exact material: where to take it, what to do before dropping it off, and what to do if local infrastructure does not support it. The webapp pulls from public recycling data matched to the customer's location and presents the result in plain, actionable language.

Beyond disposal guidance the webapp serves two additional purposes. It captures anonymized interaction data showing which components generate the most scans and which zip codes have the least local disposal infrastructure, giving Green Chef's product and operations teams a real-time signal about where packaging friction is highest. And it transforms the post-delivery experience from a source of guilt into a moment of brand reinforcement: a customer who finds a clear answer and successfully disposes of an item responsibly associates Green Chef with a brand that helped them do the right thing. This recommendation requires no changes to physical materials beyond printing QR codes on existing packaging surfaces, making it the lowest friction of the three recommendations to implement and deployable in parallel with both Tier 1 and Tier 2.

11. Acknowledgements

The team would like to express sincere gratitude to the individuals and organizations whose support made this project possible.

First and foremost we thank Kayla Baber, Product Manager at Green Chef and HelloFresh, for her guidance, availability, and willingness to share proprietary data and research with our team throughout the engagement. Her insights into Green Chef's operations, customer base, and strategic priorities were instrumental in grounding our recommendations in the realities of the business.

We thank Professor Sean Raines of the Virginia Tech Pamplin College of Business for his mentorship and feedback across the duration of this project. His direction helped sharpen both the quality of our analysis and the clarity of our final deliverables.

We also acknowledge HelloFresh and Green Chef for the opportunity to work on a meaningful and challenging business problem, and for the trust placed in a student team to engage seriously with proprietary customer and operational data under the terms of our non-disclosure agreement.

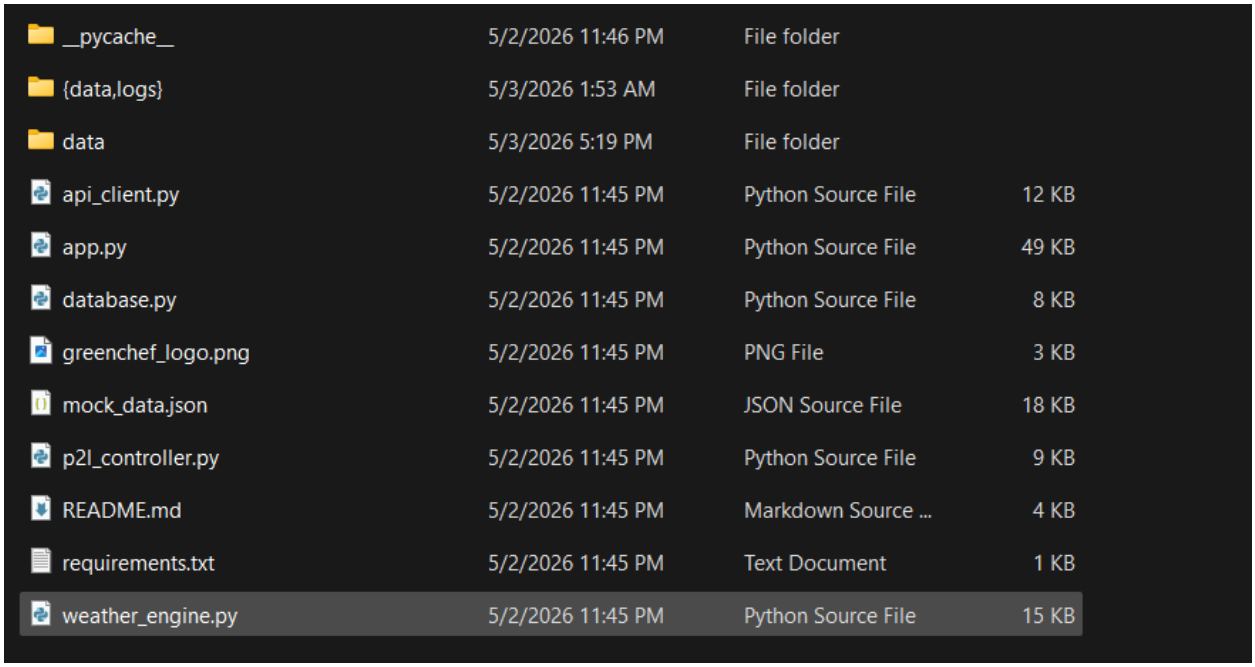
Finally we thank Virginia Tech's Masters of Science in Business Analytics program for providing the framework, resources, and academic environment that made this capstone project possible.

12. References

1. TemperPack. (n.d.). *ClimaCell insulated packaging platform*. TemperPack. https://www.temperpack.com/platforms/climacell
2. Pelton Shepherd. (n.d.). *Enviro-Ice ice packs*. Pelton Shepherd Industries. https://www.peltonshepherd.com/products/enviro-ice/
3. Pelton Shepherd. (n.d.). *Products*. Pelton Shepherd Industries. https://www.peltonshepherd.com/products/
4. Virginia Tech University Relations. (n.d.). *Brand identity: Lockups*. Virginia Tech. https://brand.vt.edu/content/brand_vt_edu/en/identity/lockups.html
5. Brandfetch. (n.d.). *Green Chef brand assets*. Brandfetch. https://brandfetch.com/greenchef.com
6. The Meal Kit Review. (n.d.). *Green Chef vs. Hungryroot: A comparison*. The Meal Kit Review. https://www.themealkitreview.com/compare/green-chef-vs-hungryroot/
7. Katana Technologies. (n.d.). *Katana cloud manufacturing software*. Katana MRP. https://katanamrp.com/
8. FLEX Assistant. (n.d.). *Pick-by-light systems: Worker assistance solutions*. FLEX Assistant. https://www.flexassistant.de/en/worker-assistance/pick-by-light-system/
9. OpenWeather. (n.d.). *OpenWeatherMap API*. OpenWeather Ltd. https://openweathermap.org
10. U.S. Food and Drug Administration. (2022). *FDA Food Code 2022*. U.S. Department of Health and Human Services. https://www.fda.gov/food/fda-food-code/food-code-2022
11. U.S. Department of Agriculture Food Safety and Inspection Service. (2023). *Danger zone (40°F–140°F)*. USDA FSIS. https://www.fsis.usda.gov/food-safety/safe-food-handling-and-preparation/food-safety-basics/danger-zone-40f-140f
12. International Safe Transit Association. (2020). *ISTA 2A: Packaged-products weighing 150 lb (68 kg) or less*. ISTA. https://www.ista.org/docs/2Adoc.pdf
13. Katana Technologies. (2024). *Katana Cloud Manufacturing API documentation*. Katana MRP. https://developer.katanamrp.com/reference

Appendix A

1. Code for the web app based smart packaging system with a weather tracker



__pycache__	5/2/2026 11:46 PM	File folder	
{data,logs}	5/3/2026 1:53 AM	File folder	
data	5/3/2026 5:19 PM	File folder	
api_client.py	5/2/2026 11:45 PM	Python Source File	12 KB
app.py	5/2/2026 11:45 PM	Python Source File	49 KB
database.py	5/2/2026 11:45 PM	Python Source File	8 KB
greenchef_logo.png	5/2/2026 11:45 PM	PNG File	3 KB
mock_data.json	5/2/2026 11:45 PM	JSON Source File	18 KB
p2l_controller.py	5/2/2026 11:45 PM	Python Source File	9 KB
README.md	5/2/2026 11:45 PM	Markdown Source ...	4 KB
requirements.txt	5/2/2026 11:45 PM	Text Document	1 KB
weather_engine.py	5/2/2026 11:45 PM	Python Source File	15 KB

a. Api_client.py

```
import json
import os
import requests
from pathlib import Path
from typing import Optional
from database import upsert_order, log

MOCK_DATA_PATH  = Path(__file__).parent / "mock_data.json"
KATANA_BASE_URL = os.getenv("KATANA_BASE_URL",
"https://api.katanamrp.com/v1")
KATANA_API_KEY  = os.getenv("KATANA_API_KEY", "")


def _load_mock() -> dict:
    with open(MOCK_DATA_PATH) as f:
        return json.load(f)


def _katana_headers() -> dict:
```

```
    return {"Authorization": f"Bearer {KATANA_API_KEY}",
"Content-Type": "application/json"}


def _fetch_live_orders() -> list[dict]:
    try:
        resp = requests.get(
            f"{KATANA_BASE_URL}/sales_orders",
            headers=_katana_headers(),
            params={"status": "open"},
            timeout=10,
        )
        resp.raise_for_status()
        raw = resp.json().get("data", [])
        return [_normalize_katana_order(o) for o in raw]
    except Exception as e:
        log("WARNING", "api_client", f"Katana live fetch failed:
{e}. Falling back to mock data.")
        return []


def _normalize_katana_order(raw: dict) -> dict:
    addr  = raw.get("shipping_address", {})
    items = []
    for line in raw.get("sales_order_rows", []):
        sku = line.get("product_variant_code", "UNK-000")
        items.append({
            "sku":       sku,
            "name":      line.get("product_name", "Unknown Item"),
            "qty":       line.get("quantity", 1),
            "weight_kg": line.get("unit_weight_kg", 0.30),
            "category":  line.get("category", ""),
            "bag_type":  line.get("bag_type", "autobagger_5x11_a"),
        })
    return {
        "order_id":         raw.get("order_no", ""),
        "customer_name":    raw.get("customer_name", ""),
        "delivery_address": addr.get("address_line_1", ""),
        "city":             addr.get("city", ""),
        "state":            addr.get("state", ""),
```

```
        "zip_code":         addr.get("zip", ""),
        "delivery_date":    raw.get("delivery_date", ""),
        "items":            items,
        "status":           "pending",
    }


# ── Bag type → catalog key mapping ──────────────────────────
BAG_TYPE_TO_CATALOG = {
    "autobagger_5x11_a": "Autobagger 5x11 Small (Paper Alt)",
    "autobagger_5x11_b": "Autobagger 5x11 Alt (Paper Alt)",
    "autobagger_8x13":   "Autobagger 8x13 (Paper Alt)",
    "none":              None,   # sauce containers, meat packaging,
spice sachets — not modelled
}


class KatanaClient:

    def __init__(self, use_mock: bool = True):
        self.use_mock = use_mock or not KATANA_API_KEY
        self._mock    = _load_mock() if self.use_mock else {}

    def fetch_open_orders(self) -> list[dict]:
        if self.use_mock:
            orders = self._mock.get("katana_orders", [])
            log("INFO", "api_client", f"Loaded {len(orders)} mock
orders from mock_data.json")
        else:
            orders = _fetch_live_orders()
            if not orders:
                orders = self._mock.get("katana_orders", [])
        for o in orders:
            upsert_order(o)
        return orders

    def get_order_by_id(self, order_id: str) -> Optional[dict]:
        orders = self.fetch_open_orders()
```

```
        match  = next((o for o in orders if o["order_id"] ==
order_id), None)
        if not match:
            log("WARNING", "api_client", f"Order {order_id} not
found.")
        return match

    def get_packing_materials_catalog(self) -> dict:
        raw = self._mock.get("packing_materials", {})
        # Strip comment-only keys (start with _)
        return {k: v for k, v in raw.items() if not
k.startswith("_")}

    def build_base_packing_list(self, order: dict) -> list[dict]:
        """
        Build the exact Green Chef box BOM for one order.

        Fixed box components (base tier — always included, no
change):
          1. Standard Box       — outer corrugated cardboard
          2. ClimaCell Liner    — insulated plant-based liner
          3. Paper Kit Bag      — groups all ingredients
          4. Recipe Card        — paper recipe instructions

        Temperature-scaled ice (Tier 1 — weather engine overrides
qty/type):
          5. Nutri-Ice Pack     — replaces Pelton Shepherd; qty set
by FDA tier

        Per-ingredient bags (Tier 2 — assigned by bag_type on each
item):
          • Autobagger 5×11.5 small  → small produce, herbs, small
dairy
          • Autobagger 5×11.5 alt    → medium produce, cheese bags
          • Autobagger 8×13.5        → protein, grains, larger items

        Items with bag_type = "none" (sauce containers, meat pkging,
spice sachets)
        are NOT assigned a bag here — no sustainable alternative is
modelled.
```

```
        """
        catalog    = self.get_packing_materials_catalog()
        packing    = []
        items      = order.get("items", [])
        categories = {i.get("category", "") for i in items}
        has_perishable = bool({"protein", "dairy", "produce",
"herb", "sauce"} & categories)

        # ── 1. Standard Box
──────────────────────────────────────────────────
        box = catalog.get("Standard Box", {})
        packing.append({
            **box,
            "name":   "Standard Box",
            "qty":    1,
            "source": "base",
            "tier":   "base",
            "note":   "Exterior corrugated cardboard — already
sustainable, no change",
        })

        # ── 2. ClimaCell Liner
──────────────────────────────────────────────────
        climacell = catalog.get("ClimaCell Liner", {})
        packing.append({
            **climacell,
            "name":   "ClimaCell Liner",
            "qty":    1,
            "source": "base",
            "tier":   "base",
            "note":   "Plant-based insulated liner, curbside
recyclable — no change needed",
        })

        # ── 3. Paper Kit Bag
──────────────────────────────────────────────────
        kit_bag = catalog.get("Paper Kit Bag", {})
        packing.append({
            **kit_bag,
            "name":   "Paper Kit Bag",
```

```
            "qty":    1,
            "source": "base",
            "tier":   "base",
            "note":   "Paper-based bag — already sustainable, no
change",
        })

        # ── 4. Recipe Card
──────────────────────────────────────────────────
        card = catalog.get("Recipe Card", {})
        packing.append({
            **card,
            "name":   "Recipe Card",
            "qty":    1,
            "source": "base",
            "tier":   "base",
            "note":   "Paper recipe card — already sustainable, no
change",
        })

        # ── 5. Nutri-Ice Pack baseline (Tier 1)
──────────────────────────────
        # The weather engine will override qty/count based on FDA
temperature tier.
        # This baseline of 1 is always included for any order with
perishables.
        if has_perishable:
            nutrice = catalog.get("Nutri-Ice Pack", {})
            packing.append({
                **nutrice,
                "name":   "Nutri-Ice Pack",
                "qty":    1,
                "source": "base",
                "tier":   "tier1",
                "note":   "Tier 1: Nutri-Ice replaces Pelton
Shepherd — weather engine adjusts qty",
            })
            log("INFO", "api_client",
                f"Order {order['order_id']}: Nutri-Ice Pack (Tier 1)
added as baseline")
```

```
        # ── 6. Autobagger bag per ingredient (Tier 2)
──────────────────────
        # Count how many of each bag variant are needed across all
ingredients.
        bag_counts: dict[str, int] = {}
        for item in items:
            bag_type = item.get("bag_type", "none")
            cat_key  = BAG_TYPE_TO_CATALOG.get(bag_type)
            if cat_key is None:
                # Sauce containers, meat packaging, spice sachets —
not modelled
                log("INFO", "api_client",
                    f"  {item['name']}: bag_type=none —
sauce/meat/spice packaging "
                    "not modelled in Tier 2 cost file")
                continue
            if cat_key not in catalog:
                log("WARNING", "api_client",
                    f"  Bag catalog entry '{cat_key}' not found for
{item['name']}")
                continue
            bag_counts[cat_key] = bag_counts.get(cat_key, 0) +
item.get("qty", 1)

        for bag_name, count in bag_counts.items():
            bag_entry = catalog.get(bag_name, {})
            packing.append({
                **bag_entry,
                "name":   bag_name,
                "qty":    count,
                "source": "tier2-bag",
                "tier":   "tier2",
                "note":   f"Tier 2: sustainable paper alternative to
clear perforated plastic bag — {bag_entry.get('description','')}",
            })

        if bag_counts:
            log("INFO", "api_client",
                f"Order {order['order_id']}: Tier 2 bags — "
```

```
                + ", ".join(f"{k} x{v}" for k, v in
bag_counts.items()))

        return packing
```

b. app.py

```
import streamlit as st
import json, base64, random
from datetime import datetime, date, timedelta
from pathlib import Path

st.set_page_config(
    page_title="Green Chef PackOps™",
    page_icon="🌿",
    layout="wide",
    initial_sidebar_state="expanded",
)

import sys
sys.path.insert(0, str(Path(__file__).parent))
from database       import init_db, get_all_orders, get_order,
save_manifest, \
                           get_recent_logs, add_manual_order,
update_order_status
from api_client     import KatanaClient
from weather_engine import (WeatherEngine, PRECIP_THRESHOLD,
                            HEAT_THRESHOLD_F, FREEZE_THRESHOLD_F,
                            EXTREME_HEAT_F, HOT_F, WARM_F,
DANGER_ZONE_MAX_F, FREEZE_RISK_F,
                            get_safety_tier)
from p2l_controller import P2LController

init_db()
katana  = KatanaClient(use_mock=True)
weather = WeatherEngine(use_mock=True)
p2l     = P2LController(protocol="simulated")
katana.fetch_open_orders()
```

```
# ── Logo (embedded base64 so no static file server needed)
──────────────────
LOGO_PATH = Path(__file__).parent / "greenchef_logo.png"
def get_logo_b64():
    try:
        with open(LOGO_PATH, "rb") as f:
            return base64.b64encode(f.read()).decode()
    except Exception:
        return ""
LOGO_B64 = get_logo_b64()
LOGO_HTML = (f'<img src="{LOGO_B64}"
style="height:44px;border-radius:6px;">'
             if LOGO_B64 else '<span
style="font-size:26px;">🌿</span>')

# ── Category icons
──────────────────────────────────────────────────────────
CAT_ICON = {
    "protein": "🥩", "produce": "🥦", "herb": "🌿", "dairy": "🧀",
    "grain": "🌾", "pantry": "🫙", "sauce": "🫕", "packaging": "📦",
}

TRANSIT_ICON = {
    "Corrugated Liner Sheet":   "📋",
    "Molded Pulp Tray":         "🫙",
    "Bubble Wrap Sheet":        "🫧",
    "Leak-Proof Bag":           "🛡️",
    "Divider Cardboard Insert": "📐",
    "Corner Foam Pad":          "🔲",
}

# ── CSS
──────────────────────────────────────────────────────────────────
───
st.markdown("""
<style>
@import
url('https://fonts.googleapis.com/css2?family=DM+Sans:wght@300;400;5
00;600;700&family=DM+Mono:wght@400;500&display=swap');
:root {
```

```
  --gc-green:       #4a7c59;
  --gc-green-mid:   #3d6b4a;
  --gc-green-dark:  #2e5438;
  --gc-green-deep:  #1e3d28;
  --gc-green-light: #7ab893;
  --gc-green-pale:  #b8d9c4;
  --gc-white:       #ffffff;
  --gc-charcoal:    #1a2820;
  --gc-forest:      #1f3028;
  --gc-bark:        #263830;
  --gc-moss:        #2e4436;
  --gc-sage:        #5a7a64;
  --gc-mist:        #8aad96;
  --gc-amber:       #e8a030;
  --gc-amber-soft:  #f5c870;
  --gc-ember:       #d94f3d;
  --gc-sky:         #4ab8d4;
  --gc-text:        #ddeee3;
  --gc-text-muted:  #7a9e86;
  --gc-border:      rgba(74,124,89,0.25);
  --gc-border-mid:  rgba(74,124,89,0.45);
  --gc-border-hi:   rgba(74,124,89,0.7);
}
html,body,[class*="css"]{font-family:'DM
Sans',sans-serif;background:var(--gc-charcoal);color:var(--gc-text);
}

/* Sidebar */
[data-testid="stSidebar"]{background:var(--gc-forest);border-right:1
px solid var(--gc-border);}
[data-testid="stSidebar"] *{color:var(--gc-text)!important;}
[data-testid="stSidebar"] .stRadio
label{font-size:14px!important;font-weight:500!important;padding:9px
14px!important;border-radius:9px!important;cursor:pointer;display:bl
ock;transition:background .15s;}
[data-testid="stSidebar"] .stRadio
label:hover{background:rgba(74,124,89,.18)!important;}

/* Cards */
```

```
.gc-card{background:var(--gc-forest);border:1px solid
var(--gc-border);border-radius:14px;padding:20px
22px;margin-bottom:14px;}
.gc-card-hdr{font-size:10px;font-weight:700;letter-spacing:.14em;tex
t-transform:uppercase;color:var(--gc-mist);margin-bottom:14px;}

/* Section titles */
.gc-title{font-size:24px;font-weight:600;color:var(--gc-white);lette
r-spacing:-.3px;margin-bottom:3px;}
.gc-sub{font-size:13px;color:var(--gc-mist);margin-bottom:20px;}

/* Weather banner */
.gc-wb{border-radius:16px;padding:22px
26px;margin-bottom:20px;display:flex;align-items:flex-start;gap:20px
;position:relative;overflow:hidden;}
.gc-wb::before{content:'';position:absolute;inset:0;background:repea
ting-linear-gradient(-45deg,rgba(255,255,255,.025)
0,rgba(255,255,255,.025) 1px,transparent 1px,transparent 8px);}
.wb-heat
{background:linear-gradient(135deg,#6b2200,#b85000);border:1px solid
#e07030;}
.wb-freeze{background:linear-gradient(135deg,#0d2440,#1a4070);border
:1px solid #5090d0;}
.wb-rain
{background:linear-gradient(135deg,#0c2030,#1a3858);border:1px solid
#3898c0;}
.wb-clear
{background:linear-gradient(135deg,var(--gc-green-deep),var(--gc-gre
en-dark));border:1px solid var(--gc-green);}
.wb-icon{font-size:56px;line-height:1;position:relative;z-index:1;fl
ex-shrink:0;}
.wb-body{flex:1;position:relative;z-index:1;}
.wb-loc{font-size:12px;color:rgba(255,255,255,.6);letter-spacing:.05
em;margin-bottom:3px;}
.wb-city{font-size:20px;font-weight:700;margin-bottom:1px;}
.wb-temp{font-size:42px;font-weight:300;font-family:'DM
Mono',monospace;line-height:1;letter-spacing:-2px;}
.wb-sup{font-size:18px;vertical-align:super;}
.wb-feels{font-size:12px;color:rgba(255,255,255,.6);margin-top:3px;}
```

```
.wb-tag{display:inline-block;margin-top:7px;margin-right:6px;font-si
ze:9px;font-weight:700;letter-spacing:.1em;text-transform:uppercase;
background:rgba(0,0,0,.35);border:1px solid
rgba(255,255,255,.18);padding:2px 9px;border-radius:20px;}

/* Forecast pills */
.fc-row{display:flex;gap:8px;flex-wrap:wrap;margin-top:12px;}
.fc-pill{background:rgba(0,0,0,.22);border:1px solid
rgba(255,255,255,.1);border-radius:11px;padding:10px
14px;text-align:center;min-width:84px;flex:1;}
.fp-date{font-size:10px;letter-spacing:.07em;text-transform:uppercas
e;color:rgba(255,255,255,.45);margin-bottom:5px;}
.fp-icon{font-size:20px;margin-bottom:4px;}
.fp-temp{font-size:19px;font-weight:600;font-family:'DM
Mono',monospace;}
.fp-rain{font-size:11px;color:var(--gc-sky);margin-top:3px;}

/* Transit protection badge */
.ci-transit{background:rgba(74,160,212,.08);border-left:3px solid
var(--gc-sky);}
.ci-badge-transit{margin-left:auto;font-size:10px;font-weight:700;le
tter-spacing:.08em;text-transform:uppercase;color:var(--gc-sky);back
ground:rgba(74,160,212,.12);padding:2px
8px;border-radius:10px;border:1px solid rgba(74,160,212,.3);}
.ci-transit-note{font-size:11px;color:var(--gc-mist);margin-top:3px;
font-style:italic;padding-left:2px;}

/* Metrics (sidebar) */
.gc-met{background:rgba(0,0,0,.2);border:1px solid
var(--gc-border);border-radius:10px;padding:11px
13px;margin-bottom:7px;text-align:center;}
.gc-met-lbl{font-size:9px;font-weight:700;letter-spacing:.12em;text-
transform:uppercase;color:var(--gc-mist);margin-bottom:4px;}
.gc-met-val{font-size:23px;font-weight:600;font-family:'DM
Mono',monospace;}

/* ═══════════════════════════════════════
   INGREDIENT PACKING CARDS — main feature
   ═══════════════════════════════════════ */
```

```
.pack-progress-bar-outer{background:rgba(0,0,0,.3);border-radius:20p
x;height:10px;margin-bottom:18px;overflow:hidden;border:1px solid
var(--gc-border);}
.pack-progress-bar-inner{height:100%;border-radius:20px;background:l
inear-gradient(90deg,var(--gc-green-mid),var(--gc-green-light));tran
sition:width .4s ease;}

/* Active card (current item to pack) */
.ing-card-active{
  background:var(--gc-forest);
  border:2px solid var(--gc-green);
  border-radius:16px;padding:22px 24px;margin-bottom:12px;
  box-shadow:0 0 24px rgba(74,124,89,.25);
  position:relative;overflow:hidden;
}
.ing-card-active::before{
  content:'';position:absolute;inset:0;
  background:linear-gradient(135deg,rgba(74,124,89,.08),transparent
60%);
  pointer-events:none;
}
/* Done card */
.ing-card-done{
  background:rgba(0,0,0,.15);
  border:1px solid rgba(74,124,89,.2);
  border-radius:12px;padding:14px 18px;margin-bottom:8px;
  opacity:.65;
}
/* Upcoming card */
.ing-card-upcoming{
  background:var(--gc-bark);
  border:1px solid var(--gc-border);
  border-radius:12px;padding:14px 18px;margin-bottom:8px;
}
.ing-row{display:flex;align-items:center;gap:14px;}
.ing-icon{font-size:28px;flex-shrink:0;}
.ing-icon-sm{font-size:20px;flex-shrink:0;}
.ing-name{font-size:16px;font-weight:600;line-height:1.2;}
.ing-name-sm{font-size:14px;font-weight:500;}
.ing-meta{font-size:12px;color:var(--gc-mist);margin-top:3px;}
```

```
.ing-qty-badge{font-family:'DM
Mono',monospace;font-size:13px;background:var(--gc-green-dark);borde
r:1px solid
var(--gc-border-hi);color:var(--gc-green-pale);padding:3px
10px;border-radius:20px;white-space:nowrap;}
.ing-bin-badge{font-family:'DM
Mono',monospace;font-size:11px;background:rgba(0,0,0,.3);border:1px
solid var(--gc-border);color:var(--gc-mist);padding:2px
8px;border-radius:6px;white-space:nowrap;}
.ing-weight-tgt{font-size:22px;font-weight:300;font-family:'DM
Mono',monospace;color:var(--gc-green-pale);}
.ing-weight-unit{font-size:13px;color:var(--gc-mist);margin-left:3px
;}
.ing-done-tick{font-size:20px;margin-left:auto;flex-shrink:0;}
.ing-step-num{font-size:11px;font-weight:700;color:var(--gc-green-li
ght);letter-spacing:.06em;margin-bottom:6px;}

/* Scale readout */
.scale-readout{
  background:rgba(0,0,0,.35);border:1px solid var(--gc-border-mid);
  border-radius:12px;padding:14px
20px;text-align:center;margin-top:14px;
}
.scale-lbl{font-size:10px;font-weight:700;letter-spacing:.1em;text-t
ransform:uppercase;color:var(--gc-mist);margin-bottom:6px;}
.scale-val{font-size:38px;font-weight:300;font-family:'DM
Mono',monospace;letter-spacing:-1px;}
.scale-unit{font-size:16px;color:var(--gc-mist);}
.scale-ok   {color:var(--gc-green-light);}
.scale-warn {color:var(--gc-amber);}
.scale-err  {color:var(--gc-ember);}

/* P2L flash badge */
.p2l-flash{
  display:inline-flex;align-items:center;gap:8px;
  background:rgba(74,124,89,.2);border:1px solid
var(--gc-border-hi);
  border-radius:8px;padding:6px 14px;margin-top:12px;
  font-size:13px;font-family:'DM Mono',monospace;
}
```

```
.p2l-dots{color:var(--gc-green-light);letter-spacing:3px;font-size:16px;}

/* Weight box final */
.gc-wb-box{border-radius:12px;padding:16px 20px;text-align:center;}
.gwb-ok   {background:rgba(46,84,56,.35);border:1px solid var(--gc-green);}
.gwb-alert{background:rgba(217,79,61,.15);border:1px solid var(--gc-ember);}
.gwb-lbl  {font-size:10px;font-weight:700;letter-spacing:.12em;text-transform:uppercase;opacity:.65;margin-bottom:7px;}
.gwb-val  {font-size:30px;font-weight:300;font-family:'DM Mono',monospace;letter-spacing:-1px;}
.gwb-sub  {font-size:12px;margin-top:5px;opacity:.7;font-family:'DM Mono',monospace;}

/* Alert banner */
.gc-alert{background:rgba(120,25,20,.55);border:2px solid var(--gc-ember);color:#fca5a5;border-radius:12px;padding:16px 22px;font-weight:700;font-size:15px;text-align:center;animation:gc-blink 1.2s step-start infinite;}
@keyframes gc-blink{0%,100%{opacity:1}50%{opacity:.55}}
@keyframes gc-pulse{0%,100%{opacity:1}50%{opacity:.3}}

/* P2L sidebar status */
.gc-p2l-pill{display:inline-flex;align-items:center;gap:7px;font-size:12px;font-weight:500;padding:5px 12px;border-radius:20px;border:1px solid var(--gc-border);background:rgba(0,0,0,.2);margin-bottom:14px;}
.dot-live{color:var(--gc-green-light);animation:gc-pulse 1.8s infinite;}
.dot-idle{color:var(--gc-sage);}

/* Buttons */
.stButton>button{background:var(--gc-green-dark)!important;color:white!important;border:none!important;border-radius:10px!important;font-family:'DM Sans',sans-serif!important;font-weight:600!important;font-size:14px!
```

```
important;padding:10px 20px!important;transition:all
.18s!important;}
.stButton>button:hover{background:var(--gc-green-mid)!important;tran
sform:translateY(-1px)!important;box-shadow:0 4px 16px
rgba(74,124,89,.35)!important;}

/* Inputs */
.stTextInput input,.stNumberInput input,.stDateInput
input{background:var(--gc-bark)!important;border:1px solid
var(--gc-border-mid)!important;color:var(--gc-text)!important;border
-radius:8px!important;font-family:'DM Sans',sans-serif!important;}
.stSelectbox>div>div{background:var(--gc-bark)!important;border:1px
solid
var(--gc-border-mid)!important;color:var(--gc-text)!important;border
-radius:8px!important;}

/* Tabs */
.stTabs
[data-baseweb="tab"]{color:var(--gc-mist)!important;font-weight:600!
important;background:transparent!important;}
.stTabs
[aria-selected="true"]{color:var(--gc-green-light)!important;border-
bottom-color:var(--gc-green)!important;}
hr{border-color:var(--gc-border)!important;}

/* Log rows */
.gc-log{font-family:'DM Mono',monospace;font-size:12px;padding:5px
10px;border-bottom:1px solid
var(--gc-border);display:flex;gap:12px;}
.ll-ts{color:var(--gc-sage);white-space:nowrap;}
.ll-INFO{color:var(--gc-green-light);}
.ll-WARNING{color:var(--gc-amber);}
.ll-ERROR{color:var(--gc-ember);}
.ll-mod{color:var(--gc-mist);}

/* Empty state */
.gc-empty{text-align:center;padding:56px 20px;color:var(--gc-sage);}
.gc-empty-icon{font-size:52px;margin-bottom:12px;}
.gc-empty-text{font-size:16px;}
```

```
/* Completion celebration */
.gc-complete{background:linear-gradient(135deg,var(--gc-green-deep),
var(--gc-green-dark));border:2px solid
var(--gc-green);border-radius:16px;padding:26px
28px;text-align:center;}
.gc-complete-icon{font-size:56px;margin-bottom:10px;}
.gc-complete-title{font-size:22px;font-weight:700;color:var(--gc-whi
te);margin-bottom:6px;}
.gc-complete-sub{font-size:14px;color:var(--gc-green-pale);}

/* Order status tag */
.status-tag{display:inline-block;font-size:10px;font-weight:700;lett
er-spacing:.08em;text-transform:uppercase;padding:2px
9px;border-radius:20px;}
.st-pending{background:rgba(232,160,48,.15);color:var(--gc-amber);bo
rder:1px solid rgba(232,160,48,.4);}
.st-packed
{background:rgba(74,124,89,.2);color:var(--gc-green-light);border:1p
x solid var(--gc-border-hi);}
.st-error
{background:rgba(217,79,61,.15);color:var(--gc-ember);border:1px
solid rgba(217,79,61,.4);}
.st-inprog
{background:rgba(74,180,212,.1);color:var(--gc-sky);border:1px solid
rgba(74,180,212,.35);}
</style>
""", unsafe_allow_html=True)


# ── Session state
─────────────────────────────────────────────────────────────
DEFAULTS = {
    "manifest": None, "weather_data": None, "active_order": None,
"alert": None,
    # Ingredient-by-ingredient packing state
    "pack_mode": False,           # True while stepping through
items
    "pack_index": 0,              # current item index (0-based)
    "pack_checked": [],           # list of booleans per item
    "pack_weights": [],           # simulated scale weight per item
```

```
    "pack_complete": False,        # all items done
    "final_weight_result": None,   # overall weight check after
completion
}
for k, v in DEFAULTS.items():
    if k not in st.session_state:
        st.session_state[k] = v


# ── Helpers
─────────────────────────────────────────────────────────────────

def wb_class(w):
    t = w.get("current_temp_f", 68)
    r = w.get("rain_chance_pct", 0)
    c = w.get("condition", "").lower()
    if t >= EXTREME_HEAT_F: return "wb-heat"
    if t >= HOT_F:          return "wb-heat"
    if t >= WARM_F:         return "wb-rain"   # amber/warm tone
    if t >= DANGER_ZONE_MAX_F: return "wb-rain"
    if t < FREEZE_RISK_F:   return "wb-freeze"
    if r > PRECIP_THRESHOLD or any(x in c for x in
["rain","snow","thunder"]): return "wb-rain"
    return "wb-clear"

def fcast_icon(cond, tf):
    c=cond.lower()
    if "snow" in c or tf<32: return "❄️"
    if "thunder" in c or "storm" in c: return "⛈️"
    if "rain" in c or "drizzle" in c: return "🌧️"
    if "cloud" in c: return "⛅"
    return "☀️"

def sim_weight(target_kg, jitter_pct=0.8):
    """Simulate a scale reading within ±jitter_pct% of target."""
    v = random.uniform(-jitter_pct/100, jitter_pct/100)
    return round(target_kg * (1 + v), 3)

def cat_icon(item):
    return CAT_ICON.get(item.get("category",""), "📦")
```

```
def all_items_for_packing(manifest):
    """Flatten meal items + packaging into one ordered list with
bin_ids."""
    items = []
    for it in manifest.get("meal_items", []):
        items.append({**it, "bin_id": it.get("bin_id", "PICK"),
"source": "meal"})
    for pk in manifest.get("packaging", []):
        items.append({**pk, "source": "packaging"})
    return items

def start_pack_mode(order_id):
    order = katana.get_order_by_id(order_id) or get_order(order_id)
    if not order:
        st.session_state.alert = f"Order **{order_id}** not found."
        return
    st.session_state.active_order   = order
    st.session_state.alert          = None
    st.session_state.pack_complete  = False
    st.session_state.final_weight_result = None

    w_data = weather.get_weather(order["zip_code"],
order.get("delivery_date",""))
    st.session_state.weather_data   = w_data

    base_packing = katana.build_base_packing_list(order)
    manifest     = weather.build_manifest(order, base_packing,
w_data)
    st.session_state.manifest       = manifest

    all_items = all_items_for_packing(manifest)
    n = len(all_items)
    st.session_state.pack_mode      = True
    st.session_state.pack_index     = 0
    st.session_state.pack_checked   = [False] * n
    st.session_state.pack_weights   = [0.0]  * n

def confirm_item(idx):
    """Mark current item as confirmed and advance."""
    items    = all_items_for_packing(st.session_state.manifest)
```

```
    item     = items[idx]
    target   = item.get("weight_kg", 0.1) * item.get("qty", 1)
    measured = sim_weight(target)
    st.session_state.pack_weights[idx]  = measured
    st.session_state.pack_checked[idx]  = True

    # flash the bin
    bin_id = item.get("bin_id", "BIN-UNK")
    qty    = item.get("qty", 1)
    p2l._send_signal(bin_id, qty)

    next_idx = idx + 1
    if next_idx >= len(items):
        st.session_state.pack_complete = True
        _finalize_pack()
    else:
        st.session_state.pack_index = next_idx

def _finalize_pack():
    manifest  = st.session_state.manifest
    order     = st.session_state.active_order
    w_data    = st.session_state.weather_data
    items     = all_items_for_packing(manifest)

    theo_kg   = manifest["weights"]["theoretical_total_kg"]
    actual_kg = round(sum(st.session_state.pack_weights), 3)
    manifest["weights"]["actual_kg"] = actual_kg
    diff_pct  = abs(theo_kg - actual_kg) / max(theo_kg, 0.001) * 100
    weight_ok = diff_pct <= 2.0
    manifest["weights"]["weight_ok"] = weight_ok

    if not weight_ok:
        p2l.flash_alert("BIN-ALL", color="RED")
        st.session_state.alert = (
            f"🚨 WEIGHT MISMATCH — LINE HALTED  |  "
            f"Expected {theo_kg:.3f} kg · Got {actual_kg:.3f} kg · Δ
{diff_pct:.1f}%"
        )
```

```
    save_manifest(order["order_id"], manifest, w_data, theo_kg,
actual_kg)
    update_order_status(order["order_id"], "packed" if weight_ok
else "error")
    st.session_state.final_weight_result = {
        "theo_kg": theo_kg, "actual_kg": actual_kg,
        "diff_pct": diff_pct, "ok": weight_ok
    }


#
═══════════════════════════════════════════════════════════════════
══════════
# SIDEBAR
#
═══════════════════════════════════════════════════════════════════
══════════
with st.sidebar:
    st.markdown(
        f"<div style='margin-bottom:6px;'>{LOGO_HTML}</div>"
        f"<div
style='font-size:10px;color:var(--gc-mist);letter-spacing:.08em;"

f"text-transform:uppercase;padding-bottom:14px;border-bottom:1px
solid var(--gc-border);'>"
        f"PackOps™ · Climate-Smart Fulfillment</div>",
        unsafe_allow_html=True
    )

    dot = "dot-live" if p2l.is_connected() else "dot-idle"
    st.markdown(
        f"<div class='gc-p2l-pill' style='margin-top:12px;'>"
        f"<span class='{dot}'>●</span>P2L  ·  "
        f"<span
style='color:var(--gc-mist)'>{p2l.get_protocol_label()}</span></div>
",
        unsafe_allow_html=True
    )
```

```
    page = st.radio("nav", ["🌿 Pack Order", "📋 Order Queue", "➕
New Order", "📊 System Logs"],
                    label_visibility="collapsed")
    st.divider()

    all_orders = get_all_orders()
    pending = sum(1 for o in all_orders if o["status"] == "pending")
    packed  = sum(1 for o in all_orders if o["status"] == "packed")
    errors  = sum(1 for o in all_orders if o["status"] == "error")

    st.markdown(
        f'<div class="gc-met"><div class="gc-met-lbl">Awaiting
Pack</div>'
        f'<div class="gc-met-val"
style="color:var(--gc-amber)">{pending}</div></div>'
        f'<div class="gc-met"><div class="gc-met-lbl">Packed
Today</div>'
        f'<div class="gc-met-val"
style="color:var(--gc-green-light)">{packed}</div></div>'
        f'<div class="gc-met"><div class="gc-met-lbl">Needs
Review</div>'
        f'<div class="gc-met-val"
style="color:var(--gc-ember)">{errors}</div></div>',
        unsafe_allow_html=True
    )
    st.markdown(
        "<div
style='margin-top:16px;font-size:9px;color:var(--gc-sage);text-align
:center;"
        "letter-spacing:.07em;'>© 2026 GREEN CHEF · CERTIFIED
ORGANIC</div>",
        unsafe_allow_html=True
    )


#
═══════════════════════════════════════════════════════════════════
══════════
# PAGE: PACK ORDER
```

```
#
═══════════════════════════════════════════════════════════════════════
══════════
if page == "🌿 Pack Order":
    st.markdown(
        "<div class='gc-title'>Pack Order</div>"
        "<div class='gc-sub'>Scan a barcode to begin — confirm each
ingredient one by one</div>",
        unsafe_allow_html=True
    )

    # Alert
    if st.session_state.alert:
        if "WEIGHT MISMATCH" in str(st.session_state.alert):
            st.markdown(f'<div
class="gc-alert">{st.session_state.alert}</div>',
unsafe_allow_html=True)
        else:
            st.error(st.session_state.alert)
        st.markdown("<br>", unsafe_allow_html=True)

    # ── Scan bar (only shown before pack mode starts)
──────────────────────
    if not st.session_state.pack_mode:
        c1, c2 = st.columns([4, 1])
        with c1:
            scan_id = st.text_input("Order ID", placeholder="Scan
barcode or type e.g. GC-2026-0501",
                                    label_visibility="collapsed",
key="scan_input")
        with c2:
            if st.button("▶  Begin Pack", use_container_width=True):
                if scan_id.strip():
                    with st.spinner("Fetching weather & building
manifest…"):
                        start_pack_mode(scan_id.strip().upper())
                    st.rerun()

        # Quick load chips
        demos = get_all_orders()[:5]
```

```
        if demos:
            st.markdown(
                "<div
style='font-size:10px;letter-spacing:.1em;text-transform:uppercase;"

"color:var(--gc-mist);margin-top:8px;margin-bottom:6px;'>Quick
Load</div>",
                unsafe_allow_html=True
            )
            qcols = st.columns(min(len(demos), 5))
            for i, o in enumerate(demos):
                with qcols[i]:
                    lbl = f"🌿 {o['order_id']}"
                    if st.button(lbl, key=f"q_{o['order_id']}"):
                        with st.spinner(f"Loading
{o['order_id']}…"):
                            start_pack_mode(o["order_id"])
                        st.rerun()

        st.markdown(
            "<div class='gc-empty'><div
class='gc-empty-icon'>🌿</div>"
            "<div class='gc-empty-text'>Scan or select an order
above to begin packing</div></div>",
            unsafe_allow_html=True
        )

    # ── ACTIVE PACKING MODE
──────────────────────────────────────────────

    else:
        manifest  = st.session_state.manifest
        order     = st.session_state.active_order
        w_data    = st.session_state.weather_data
        all_items = all_items_for_packing(manifest)
        n         = len(all_items)
        idx       = st.session_state.pack_index
        checked   = st.session_state.pack_checked
        complete  = st.session_state.pack_complete

        col_left, col_right = st.columns([7, 3], gap="large")
```

```
        with col_left:
            # ── Weather Banner
──────────────────────────────────────────────────
            wc    = wb_class(w_data)
            icon  = weather.get_weather_icon(w_data)
            temp  = w_data["current_temp_f"]
            feels = w_data.get("feels_like_f", temp)
            city  = w_data.get("city", order["city"])
            rain  = w_data["rain_chance_pct"]
            tags  = []
            eff_temp = max(temp, max((f.get("high_f", temp) for f in
w_data.get("forecast",[])), default=temp))
            tier  = get_safety_tier(eff_temp)
            if eff_temp >= EXTREME_HEAT_F:
                tags.append(f"🌡️ EXTREME HEAT — 3× XL Ice + Foil
Liner (FDA §3-501.11)")
            elif eff_temp >= HOT_F:
                tags.append(f"☀️ HOT {eff_temp:.0f}°F — 2× XL Ice
Pack + Foil Liner (FDA §3-501.11)")
            elif eff_temp >= WARM_F:
                tags.append(f"🌤️ WARM {eff_temp:.0f}°F — 2× Standard
Ice Pack required (FDA danger zone)")
            elif eff_temp >= DANGER_ZONE_MAX_F:
                tags.append(f"⚠️ {eff_temp:.0f}°F ≥ 40°F FDA
threshold — 1× Ice Pack mandatory")
            elif eff_temp < FREEZE_RISK_F:
                tags.append(f"❄️ FREEZE RISK {eff_temp:.0f}°F —
Insulation + Freeze Guard Liner")
            else:
                tags.append(f"✅ {eff_temp:.0f}°F — Below FDA danger
zone · Insulation sleeve only")
            if rain > PRECIP_THRESHOLD:    tags.append(f"🌧️ {rain}%
Precip — Waterproof Wrap Added")
            if w_data.get("is_fallback"):  tags.append("⚠️ Weather
API Offline — Conservative Pack Applied")
            tags_html = "".join(f"<span class='wb-tag'>{t}</span>"
for t in tags)

            st.markdown(f"""
```

```
            <div class="gc-wb {wc}">
              <div class="wb-icon">{icon}</div>
              <div class="wb-body">
                <div class="wb-loc">📍 {order['delivery_address']},
{order['city']}, {order['state']} {order['zip_code']}</div>
                <div class="wb-city">{city}</div>
                <div class="wb-temp">{temp:.0f}<span
class="wb-sup">°F</span>
                  <span
style="font-size:16px;font-weight:300;opacity:.65;"> · feels
{feels:.0f}°F · {w_data['condition']}</span>
                </div>
                <div>{tags_html}</div>
              </div>
            </div>
            """, unsafe_allow_html=True)

            # Forecast pills
            forecast = w_data.get("forecast", [])
            if forecast:
                pills = "<div class='fc-row'>"
                for day in forecast[:5]:
                    try:    label =
datetime.strptime(day["date"],"%Y-%m-%d").strftime("%a %-d")
                    except: label = day.get("date","")
                    fi = fcast_icon(day.get("condition",""),
day.get("high_f",68))
                    pills += (f"<div class='fc-pill'><div
class='fp-date'>{label}</div>"
                              f"<div class='fp-icon'>{fi}</div>"
                              f"<div
class='fp-temp'>{day.get('high_f','--'):.0f}°</div>"
                              f"<div
class='fp-rain'>💧{day.get('rain_chance_pct',0)}%</div></div>")
                pills += "</div>"
                st.markdown(pills, unsafe_allow_html=True)
                st.markdown("<br>", unsafe_allow_html=True)

            # ── FDA Food Safety Compliance Panel
──────────────────────────────
```

```
            tier_colors = {
                "heat":   ("#5c2000", "#f97316", "🌡️"),
                "warn":   ("#4a2e00", "#f59e0b", "⚠️"),
                "safe":   ("#1b3d22", "#4caf50", "✅"),
                "freeze": ("#0d2440", "#60a5fa", "❄️"),
            }
            bg, border, ticon = tier_colors.get(tier["color"],
("#1f3028","#4a7c59","ℹ️"))
            st.markdown(
                f"<div style='background:{bg};border:1px solid
{border};border-radius:12px;"
                f"padding:13px 17px;margin-bottom:18px;'>"
                f"<div
style='font-size:9px;font-weight:700;letter-spacing:.13em;"

f"text-transform:uppercase;color:{border};margin-bottom:5px;'>"
                f"{ticon}  FDA Food Safety Status ·
{tier['tier']}</div>"
                f"<div
style='font-size:12px;color:var(--gc-text);line-height:1.6;'>"
                f"{tier['fda_note']}</div>"
                f"<div
style='font-size:10px;color:var(--gc-mist);margin-top:5px;'>"
                f"Ref: FDA Food Code 2022 §3-501.11 · USDA FSIS
Danger Zone 40°F – 140°F</div>"
                f"</div>",
                unsafe_allow_html=True
            )

            # ── PROGRESS BAR
──────────────────────────────────────────────

            done_count = sum(checked)
            pct        = int(done_count / n * 100)
            st.markdown(
                f"<div
style='display:flex;justify-content:space-between;align-items:center
;"

f"margin-bottom:6px;font-size:12px;color:var(--gc-mist);'>"
                f"<span>Packing progress</span>"
```

```
                f"<span style='font-family:DM
Mono,monospace;color:var(--gc-green-light);'>"
                f"{done_count} / {n} items confirmed</span></div>"
                f"<div class='pack-progress-bar-outer'>"
                f"<div class='pack-progress-bar-inner'
style='width:{pct}%;'></div></div>",
                unsafe_allow_html=True
            )

            # ── COMPLETION STATE
─────────────────────────────────────────────
            if complete:
                wr = st.session_state.final_weight_result
                if wr and wr["ok"]:
                    st.markdown(
                        f"<div class='gc-complete'>"
                        f"<div class='gc-complete-icon'>✅</div>"
                        f"<div class='gc-complete-title'>Order
Packed Successfully!</div>"
                        f"<div class='gc-complete-sub'>"
                        f"All {n} items confirmed · Total weight
{wr['actual_kg']:.3f} kg · "
                        f"Variance {wr['diff_pct']:.1f}% ✓</div>"
                        f"</div>",
                        unsafe_allow_html=True
                    )
                st.markdown("<br>", unsafe_allow_html=True)
                if st.button("📦 Pack Another Order"):
                    for k, v in DEFAULTS.items():
                        st.session_state[k] = v
                    st.rerun()

            # ── ACTIVE ITEM CARD
─────────────────────────────────────────────
            elif not complete:
                item    = all_items[idx]
                target  = round(item.get("weight_kg", 0.1) *
item.get("qty", 1), 3)
                bin_id  = item.get("bin_id", "PICK")
                icon_i  = cat_icon(item)
```

```
                st.markdown(
                    f"<div class='ing-step-num'>STEP {idx+1} OF
{n}</div>"
                    f"<div class='ing-card-active'>"
                    f"<div class='ing-row'>"
                    f"  <div class='ing-icon'>{icon_i}</div>"
                    f"  <div style='flex:1;'>"
                    f"    <div
class='ing-name'>{item['name']}</div>"
                    f"    <div
class='ing-meta'>{item.get('category','').capitalize()} "
                    f"· {item.get('unit','piece')}</div>"
                    f"  </div>"
                    f"  <div
style='display:flex;flex-direction:column;align-items:flex-end;gap:6
px;'>"
                    f"    <span class='ing-qty-badge'>×
{item.get('qty',1)}</span>"
                    f"    <span
class='ing-bin-badge'>{bin_id}</span>"
                    f"  </div>"
                    f"</div>"
                    f"<div class='scale-readout'>"
                    f"  <div class='scale-lbl'>🔴 Place on Scale —
Target Weight</div>"
                    f"  <div class='scale-val scale-ok'>{target:.3f}
<span class='scale-unit'>kg</span></div>"
                    f"  <div
style='font-size:11px;color:var(--gc-mist);margin-top:4px;'>Toleranc
e ±2%  ·  "
                    f"{target*0.98:.3f} – {target*1.02:.3f} kg
acceptable</div>"
                    f"</div>"
                    f"<div class='p2l-flash'>"
                    f"  <span>💡 BIN {bin_id}</span>"
                    f"  <span class='p2l-dots'>{'●' *
min(item.get('qty',1),6)}</span>"
```

```
                    f"  <span
style='color:var(--gc-mist);font-size:11px;'>Flash ×
{item.get('qty',1)}</span>"
                    f"</div>"
                    f"</div>",
                    unsafe_allow_html=True
                )

                c_btn1, c_btn2 = st.columns([2, 1])
                with c_btn1:
                    if st.button(f"✅  Confirm — {item['name']}",
use_container_width=True, key=f"confirm_{idx}"):
                        confirm_item(idx)
                        st.rerun()
                with c_btn2:
                    if st.button("⏭ Skip Item",
use_container_width=True, key=f"skip_{idx}"):
                        st.session_state.pack_checked[idx] = False
                        nxt = idx + 1
                        if nxt >= n:
                            st.session_state.pack_complete = True
                            _finalize_pack()
                        else:
                            st.session_state.pack_index = nxt
                        st.rerun()

            # ── DONE & UPCOMING ITEMS
──────────────────────────────────────

            st.markdown("<br><div class='gc-card-hdr'>All
Items</div>", unsafe_allow_html=True)
            for i, item in enumerate(all_items):
                icon_i = cat_icon(item)
                wt     = round(item.get("weight_kg",0.1) *
item.get("qty",1), 3)
                tier   = item.get("tier", item.get("source", ""))
                is_t1  = "tier1" in tier
                is_t2  = tier == "tier2"
                type_label = ("🧊 Tier 1" if is_t1 else ("🌱 Tier 2"
if is_t2 else ""))
```

```
                tcolor     = "var(--gc-amber)" if is_t1 else
("var(--gc-green-light)" if is_t2 else "")

                if checked[i]:
                    meas = st.session_state.pack_weights[i]
                    badge_html = (f" <span
style='font-size:10px;color:{tcolor};'>{type_label}</span>"
                                  if type_label else "")
                    st.markdown(
                        f"<div class='ing-card-done'><div
class='ing-row'>"
                        f"<div class='ing-icon-sm'>{icon_i}</div>"
                        f"<div style='flex:1;'>"
                        f"<div
class='ing-name-sm'>{item['name']}{badge_html}</div>"
                        f"<div
class='ing-meta'>{item.get('bin_id','—')} · measured {meas:.3f}
kg</div>"
                        f"</div>"
                        f"<div class='ing-done-tick'>✅</div>"
                        f"</div></div>",
                        unsafe_allow_html=True
                    )
                elif i == idx and not complete:
                    pass  # shown above as active card
                else:
                    note = item.get("note", item.get("fda_rule",
""))
                    badge_html = (f" <span
style='font-size:10px;color:{tcolor};opacity:.8;'>{type_label}</span
>"
                                  if type_label else "")
                    st.markdown(
                        f"<div class='ing-card-upcoming'><div
class='ing-row'>"
                        f"<div class='ing-icon-sm'
style='opacity:.5;'>{icon_i}</div>"
                        f"<div style='flex:1;'>"
                        f"<div class='ing-name-sm'
style='opacity:.7;'>{item['name']}{badge_html}</div>"
```

```
                        f"<div
class='ing-meta'>{item.get('bin_id','—')} · {wt:.3f} kg
expected</div>"
                        f"{'<div class=\"ci-transit-note\">' +
note[:80] + ('…' if len(note)>80 else '') + '</div>' if note else
''}"
                        f"</div>"
                        f"<span class='ing-qty-badge'
style='opacity:.5;'>× {item.get('qty',1)}</span>"
                        f"</div></div>",
                        unsafe_allow_html=True
                    )

        with col_right:
            # Order card
            st.markdown(
                f"<div class='gc-card'>"
                f"<div class='gc-card-hdr'>Order Details</div>"
                f"<div
style='font-size:20px;font-weight:700;letter-spacing:-.3px;margin-bo
ttom:2px;'>"
                f"{order['order_id']}</div>"
                f"<div
style='font-size:15px;font-weight:500;margin-bottom:3px;'>{order['cu
stomer_name']}</div>"
                f"<div
style='font-size:12px;color:var(--gc-mist);line-height:1.6;'>"
                f"{order['delivery_address']}<br>{order['city']},
{order['state']} {order['zip_code']}</div>"
                f"<div
style='margin-top:10px;font-size:13px;color:var(--gc-amber);font-wei
ght:500;'>"
                f"📅 Delivery:
{order.get('delivery_date','')}</div>"
                f"</div>",
                unsafe_allow_html=True
            )

            # Final weight card (after completion)
            wr = st.session_state.final_weight_result
```

```
            if wr:
                ok     = wr["ok"]
                wclass = "gwb-ok" if ok else "gwb-alert"
                vcolor = "var(--gc-green-light)" if ok else
"var(--gc-ember)"
                st.markdown(
                    f"<div class='gc-wb-box {wclass}'>"
                    f"<div class='gwb-lbl'>{'✅ Weight Check · PASS'
if ok else '🚨 Weight Check · FAIL'}</div>"
                    f"<div class='gwb-val'
style='color:{vcolor};'>{wr['actual_kg']:.3f} kg</div>"
                    f"<div class='gwb-sub'>Expected
{wr['theo_kg']:.3f} kg · Δ {wr['diff_pct']:.1f}%</div>"
                    f"</div>",
                    unsafe_allow_html=True
                )
                st.markdown("<br>", unsafe_allow_html=True)

            # Packaging summary card
            st.markdown("<div class='gc-card-hdr'>Packaging
Summary</div>", unsafe_allow_html=True)
            pkg_groups = {
                "📦 Base (unchanged)":       [p for p in
manifest.get("packaging",[]) if p.get("tier") == "base"],
                "🧊 Tier 1 — Nutri-Ice":     [p for p in
manifest.get("packaging",[]) if p.get("tier") in
("tier1","tier1-fda")],
                "🌱 Tier 2 — Paper Bags":    [p for p in
manifest.get("packaging",[]) if p.get("tier") == "tier2"],
            }
            for group_label, pkg_list in pkg_groups.items():
                if not pkg_list:
                    continue
                st.markdown(
                    f"<div
style='font-size:9px;font-weight:700;letter-spacing:.1em;text-transf
orm:uppercase;"
                    f"color:var(--gc-mist);margin:10px 0
5px;'>{group_label}</div>",
                    unsafe_allow_html=True
```

```
                )
                for pkg in pkg_list:
                    tier   = pkg.get("tier", "")
                    is_t1  = "tier1" in tier
                    is_t2  = tier == "tier2"
                    color  = "var(--gc-amber)" if is_t1 else
("var(--gc-green-light)" if is_t2 else "var(--gc-mist)")
                    note   = pkg.get("note", pkg.get("fda_rule",
""))
                    st.markdown(
                        f"<div style='padding:7px
0;border-bottom:1px solid var(--gc-border);font-size:13px;'>"
                        f"<div
style='display:flex;align-items:center;gap:8px;'>"
                        f"<span style='font-family:DM
Mono,monospace;font-size:10px;color:var(--gc-mist);'>{pkg.get('bin_i
d','—')}</span>"
                        f"<span style='flex:1;'>📦
{pkg['name']}</span>"
                        f"<span style='font-family:DM
Mono,monospace;font-size:12px;color:{color};'>×{pkg['qty']}</span>"
                        f"</div>"
                        f"{'<div
style=\"font-size:11px;color:var(--gc-mist);margin-top:2px;font-styl
e:italic;\">' + note[:80] + ('…' if len(note)>80 else '') + '</div>'
if note else ''}"
                        f"</div>",
                        unsafe_allow_html=True
                    )

            # Abort button
            st.markdown("<br>", unsafe_allow_html=True)
            if st.button("✕  Abort & Rescan",
use_container_width=True):
                for k, v in DEFAULTS.items():
                    st.session_state[k] = v
                st.rerun()

            with st.expander("🗂️ Manifest JSON"):
```

```
                st.code(json.dumps(manifest, indent=2),
language="json")



#
═══════════════════════════════════════════════════════════════════
═══════════
# PAGE: ORDER QUEUE
#
═══════════════════════════════════════════════════════════════════
═══════════
elif page == "📋 Order Queue":
    st.markdown(
        "<div class='gc-title'>Order Queue</div>"
        "<div class='gc-sub'>May 2026 · All Green Chef orders synced
from Katana ERP</div>",
        unsafe_allow_html=True
    )
    all_orders = get_all_orders()
    if not all_orders:
        st.markdown("<div class='gc-empty'><div
class='gc-empty-icon'>📋</div>"
                    "<div class='gc-empty-text'>No orders
found.</div></div>", unsafe_allow_html=True)
    else:
        sf = st.selectbox("Filter", ["All", "pending", "packed",
"error"])
        filtered = all_orders if sf == "All" else [o for o in
all_orders if o["status"] == sf]

        for o in filtered:
            smap =
{"pending":("st-pending","🟡"),"packed":("st-packed","✅"),"error":(
"st-error","🔴")}
            scls, sico = smap.get(o["status"],("st-pending","⚪"))
            with st.expander(f"{sico}  {o['order_id']}  ·
{o['customer_name']}  ·  "
                             f"{o['city']}, {o['state']}  ·  📅
{o['delivery_date']}"):
                c1,c2,c3 = st.columns(3)
```

```
                with c1:
                    st.markdown(f"**ZIP**   `{o['zip_code']}`")
                    st.markdown(f"**Items**  
{len(o['items'])}")
                with c2:
                    st.markdown(
                        f"**Status**   <span class='status-tag
{scls}'>{o['status'].upper()}</span>",
                        unsafe_allow_html=True
                    )
                with c3:
                    if st.button("🌿 Pack Now",
key=f"pack_{o['order_id']}"):
                        with st.spinner("Loading…"):
                            start_pack_mode(o["order_id"])
                        st.rerun()


#
═══════════════════════════════════════════════════════════════════
══════════
# PAGE: NEW ORDER
#
═══════════════════════════════════════════════════════════════════
══════════
elif page == "➕ New Order":
    st.markdown(
        "<div class='gc-title'>New Order</div>"
        "<div class='gc-sub'>Register a manual order — climate
packaging applied at pack time</div>",
        unsafe_allow_html=True
    )
    with st.form("new_order_form", clear_on_submit=True):
        c1,c2 = st.columns(2)
        with c1:
            order_id      = st.text_input("Order ID *",
placeholder="GC-2026-0510")
            customer_name = st.text_input("Customer Name *",
placeholder="Alex Johnson")
```

```
            delivery_date = st.date_input("Delivery Date *",
value=date(2026,5,20))
        with c2:
            address  = st.text_input("Street Address *",
placeholder="123 Elm St")
            city     = st.text_input("City *", placeholder="Denver")
            cs1,cs2  = st.columns(2)
            with cs1: state    = st.text_input("State",
placeholder="CO", max_chars=2)
            with cs2: zip_code = st.text_input("ZIP *",
placeholder="80202")

        st.divider()
        st.markdown("**Meal Kit Ingredients**")
        num_items = st.number_input("Number of items", min_value=1,
max_value=15, value=4)
        items = []
        for i in range(int(num_items)):
            ic1,ic2,ic3,ic4 = st.columns([4,1,1,2])
            with ic1: iname = st.text_input(f"Item {i+1}",
key=f"in_{i}", placeholder="Chicken Thigh")
            with ic2: iqty  = st.number_input("Qty",   min_value=1,
max_value=20, key=f"iq_{i}", value=2)
            with ic3: iwt   = st.number_input("kg",
min_value=0.005, max_value=5.0, key=f"iw_{i}", value=0.25,
step=0.005)
            with ic4: icat  = st.selectbox("Category",
["protein","produce","herb","dairy","grain","pantry","sauce"],
key=f"ic_{i}")
            if iname:

items.append({"sku":f"GC-MAN-{i+1:03d}","name":iname,"qty":iqty,"wei
ght_kg":iwt,"category":icat,"unit":"piece"})

        submitted = st.form_submit_button("🌿 Create Order",
use_container_width=True)

    if submitted:
        if not all([order_id, customer_name, address, city,
zip_code]):
```

```
            st.error("Please fill in all required fields (*)")
        elif not items:
            st.error("Add at least one ingredient.")
        else:
            add_manual_order({

"order_id":order_id.strip().upper(),"customer_name":customer_name.st
rip(),

"delivery_address":address.strip(),"city":city.strip(),

"state":state.strip().upper(),"zip_code":zip_code.strip(),

"delivery_date":str(delivery_date),"items":items,"status":"pending",
            })
            st.success(f"✅ Order **{order_id.upper()}** created —
go to Pack Order to process it.")



#
═══════════════════════════════════════════════════════════════════
══════════
# PAGE: SYSTEM LOGS
#
═══════════════════════════════════════════════════════════════════
══════════
elif page == "📊 System Logs":
    st.markdown(
        "<div class='gc-title'>System Logs</div>"
        "<div class='gc-sub'>Real-time PackOps™ event stream</div>",
        unsafe_allow_html=True
    )
    logs = get_recent_logs(150)
    lf   = st.multiselect("Levels", ["INFO","WARNING","ERROR"],
default=["INFO","WARNING","ERROR"])
    logs = [l for l in logs if l["level"] in lf]
    st.markdown(f"<div
style='font-size:11px;color:var(--gc-mist);margin-bottom:8px;'>"
                f"{len(logs)} entries</div>",
unsafe_allow_html=True)
```

```
    for e in logs:
        lv = e["level"]
        st.markdown(
            f"<div class='gc-log'>"
            f"<span class='ll-ts'>{e['logged_at'][:19]}</span>"
            f"<span class='ll-{lv}'>{lv}</span>"
            f"<span class='ll-mod'>[{e['module']}]</span>"
            f"<span>{e['message']}</span></div>",
            unsafe_allow_html=True
        )
    if not logs:
        st.markdown("<div class='gc-empty'><div
class='gc-empty-icon'>📊</div>"
                    "<div class='gc-empty-text'>No entries match
this filter.</div></div>",
                    unsafe_allow_html=True)
    if st.button("🔄 Refresh"):
        st.rerun()
```

c. database.py

```
import sqlite3
import json
import os
from datetime import datetime
from pathlib import Path

DB_PATH = Path(__file__).parent / "data" / "fulfillment.db"


def get_conn():
    DB_PATH.parent.mkdir(exist_ok=True)
    conn = sqlite3.connect(str(DB_PATH), check_same_thread=False)
    conn.row_factory = sqlite3.Row
    return conn


def init_db():
    conn = get_conn()
    c = conn.cursor()
```

```
c.executescript("""
    CREATE TABLE IF NOT EXISTS orders (
        id              INTEGER PRIMARY KEY AUTOINCREMENT,
        order_id        TEXT UNIQUE NOT NULL,
        customer_name   TEXT NOT NULL,
        delivery_address TEXT NOT NULL,
        city            TEXT NOT NULL,
        state           TEXT NOT NULL,
        zip_code        TEXT NOT NULL,
        delivery_date   TEXT NOT NULL,
        items_json      TEXT NOT NULL,
        status          TEXT DEFAULT 'pending',
        created_at      TEXT DEFAULT (datetime('now')),
        updated_at      TEXT DEFAULT (datetime('now'))
    );

    CREATE TABLE IF NOT EXISTS manifests (
        id              INTEGER PRIMARY KEY AUTOINCREMENT,
        order_id        TEXT NOT NULL,
        manifest_json   TEXT NOT NULL,
        weather_json    TEXT NOT NULL,
        theoretical_weight_kg REAL,
        actual_weight_kg REAL,
        weight_ok       INTEGER DEFAULT 1,
        packed_at       TEXT DEFAULT (datetime('now')),
        packed_by       TEXT DEFAULT 'worker',
        FOREIGN KEY(order_id) REFERENCES orders(order_id)
    );

    CREATE TABLE IF NOT EXISTS system_logs (
        id              INTEGER PRIMARY KEY AUTOINCREMENT,
        level           TEXT NOT NULL,
        module          TEXT NOT NULL,
        message         TEXT NOT NULL,
        logged_at       TEXT DEFAULT (datetime('now'))
    );

    CREATE TABLE IF NOT EXISTS p2l_events (
        id              INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
            order_id        TEXT NOT NULL,
            bin_id          TEXT NOT NULL,
            item_name       TEXT NOT NULL,
            flash_count     INTEGER NOT NULL,
            triggered_at    TEXT DEFAULT (datetime('now'))
        );
    """)
    conn.commit()
    conn.close()


# ── Orders ──────────────────────────────────────────────────────────


def upsert_order(order: dict):
    conn = get_conn()
    c = conn.cursor()
    c.execute("""
        INSERT INTO orders (order_id, customer_name,
delivery_address, city, state,
                            zip_code, delivery_date, items_json)
        VALUES (:order_id, :customer_name, :delivery_address, :city,
:state,
                :zip_code, :delivery_date, :items_json)
        ON CONFLICT(order_id) DO UPDATE SET
            customer_name    = excluded.customer_name,
            delivery_address = excluded.delivery_address,
            city             = excluded.city,
            state            = excluded.state,
            zip_code         = excluded.zip_code,
            delivery_date    = excluded.delivery_date,
            items_json       = excluded.items_json,
            updated_at       = datetime('now')
    """, {**order, "items_json": json.dumps(order.get("items",
[]))})
    conn.commit()
    conn.close()


def get_all_orders():
```

```
    conn = get_conn()
    rows = conn.execute(
        "SELECT * FROM orders ORDER BY created_at DESC"
    ).fetchall()
    conn.close()
    result = []
    for r in rows:
        d = dict(r)
        d["items"] = json.loads(d["items_json"])
        result.append(d)
    return result


def get_order(order_id: str):
    conn = get_conn()
    row = conn.execute(
        "SELECT * FROM orders WHERE order_id = ?", (order_id,)
    ).fetchone()
    conn.close()
    if not row:
        return None
    d = dict(row)
    d["items"] = json.loads(d["items_json"])
    return d


def update_order_status(order_id: str, status: str):
    conn = get_conn()
    conn.execute(
        "UPDATE orders SET status = ?, updated_at = datetime('now')
WHERE order_id = ?",
        (status, order_id)
    )
    conn.commit()
    conn.close()


def add_manual_order(order_data: dict):
    """Insert a manually entered order from the UI."""
    upsert_order(order_data)
```

```
# ── Manifests ──────────────────────────────────────────────────────────

def save_manifest(order_id: str, manifest: dict, weather: dict,
                  theoretical_kg: float, actual_kg: float):
    weight_ok = abs(theoretical_kg - actual_kg) /
max(theoretical_kg, 0.001) <= 0.02
    conn = get_conn()
    conn.execute("""
        INSERT INTO manifests (order_id, manifest_json,
weather_json,
                               theoretical_weight_kg,
actual_weight_kg, weight_ok)
        VALUES (?, ?, ?, ?, ?, ?)
    """, (order_id, json.dumps(manifest), json.dumps(weather),
          theoretical_kg, actual_kg, int(weight_ok)))
    conn.commit()
    conn.close()
    return weight_ok


def get_manifests_for_order(order_id: str):
    conn = get_conn()
    rows = conn.execute(
        "SELECT * FROM manifests WHERE order_id = ? ORDER BY
packed_at DESC",
        (order_id,)
    ).fetchall()
    conn.close()
    result = []
    for r in rows:
        d = dict(r)
        d["manifest"] = json.loads(d["manifest_json"])
        d["weather"]  = json.loads(d["weather_json"])
        result.append(d)
    return result
```

```
# ── Logs ──────────────────────────────────────────────────────────

def log(level: str, module: str, message: str):
    conn = get_conn()
    conn.execute(
        "INSERT INTO system_logs (level, module, message) VALUES (?,
?, ?)",
        (level.upper(), module, message)
    )
    conn.commit()
    conn.close()
    print(f"[{level.upper()}] [{module}] {message}")


def get_recent_logs(n: int = 50):
    conn = get_conn()
    rows = conn.execute(
        "SELECT * FROM system_logs ORDER BY logged_at DESC LIMIT ?",
(n,)
    ).fetchall()
    conn.close()
    return [dict(r) for r in rows]


# ── P2L Events ────────────────────────────────────────────────────

def log_p2l_event(order_id: str, bin_id: str, item_name: str,
flash_count: int):
    conn = get_conn()
    conn.execute("""
        INSERT INTO p2l_events (order_id, bin_id, item_name,
flash_count)
        VALUES (?, ?, ?, ?)
    """, (order_id, bin_id, item_name, flash_count))
    conn.commit()
    conn.close()
```

```
def get_p2l_events(order_id: str):
    conn = get_conn()
    rows = conn.execute(
        "SELECT * FROM p2l_events WHERE order_id = ? ORDER BY
triggered_at",
        (order_id,)
    ).fetchall()
    conn.close()
    return [dict(r) for r in rows]
```

d. p2l_controller.py

```
import json
import time
import threading
from datetime import datetime
from typing import Optional
from database import log, log_p2l_event

# Optional MQTT support
try:
    import paho.mqtt.client as mqtt
    MQTT_AVAILABLE = True
except ImportError:
    MQTT_AVAILABLE = False

MQTT_BROKER   = "localhost"
MQTT_PORT     = 1883
MQTT_TOPIC_BASE = "packing/floor"

# Modbus optional
try:
    from pymodbus.client import ModbusTcpClient
    MODBUS_AVAILABLE = True
except ImportError:
    MODBUS_AVAILABLE = False

MODBUS_HOST = "192.168.1.100"
MODBUS_PORT = 502
```

```python
class P2LStatus:
    """Shared mutable state for the UI status indicator."""
    def __init__(self):
        self.active       = False
        self.current_bin  = ""
        self.current_item = ""
        self.flash_queue: list[dict] = []
        self.completed:   list[dict] = []
        self.error_msg    = ""
        self._lock        = threading.Lock()

    def set_active(self, bin_id: str, item: str):
        with self._lock:
            self.active       = True
            self.current_bin  = bin_id
            self.current_item = item

    def set_idle(self):
        with self._lock:
            self.active       = False
            self.current_bin  = ""
            self.current_item = ""

    def add_completed(self, event: dict):
        with self._lock:
            self.completed.append(event)

    def reset(self):
        with self._lock:
            self.active       = False
            self.current_bin  = ""
            self.current_item = ""
            self.flash_queue  = []
            self.completed    = []
            self.error_msg    = ""


# Singleton status for Streamlit to read
p2l_status = P2LStatus()
```

```
class P2LController:
    """
    Coordinates Pick-to-Light signals for a packing manifest.
    Supports: MQTT (preferred), Modbus TCP, or simulated mode.
    """

    def __init__(self, protocol: str = "simulated"):
        self.protocol = protocol
        self._mqtt_client: Optional[object] = None
        self._modbus_client: Optional[object] = None
        self._connected = False

        if protocol == "mqtt" and MQTT_AVAILABLE:
            self._init_mqtt()
        elif protocol == "modbus" and MODBUS_AVAILABLE:
            self._init_modbus()
        else:
            self.protocol = "simulated"
            log("INFO", "p2l_controller",
                "Running in SIMULATED mode — no physical P2L
hardware required.")

    # ── Init ──────────────────────────────────────────────────────────

    def _init_mqtt(self):
        try:
            self._mqtt_client =
mqtt.Client(client_id="p2l_fulfillment")
            self._mqtt_client.on_connect    = self._on_mqtt_connect
            self._mqtt_client.on_disconnect =
self._on_mqtt_disconnect
            self._mqtt_client.connect(MQTT_BROKER, MQTT_PORT,
keepalive=60)
            self._mqtt_client.loop_start()
            self._connected = True
            log("INFO", "p2l_controller", f"MQTT connected to
{MQTT_BROKER}:{MQTT_PORT}")
```

```
        except Exception as e:
            log("ERROR", "p2l_controller", f"MQTT init failed: {e}.
Falling back to simulated.")
            self.protocol = "simulated"

    def _init_modbus(self):
        try:
            self._modbus_client = ModbusTcpClient(MODBUS_HOST,
port=MODBUS_PORT)
            if self._modbus_client.connect():
                self._connected = True
                log("INFO", "p2l_controller", f"Modbus TCP connected
to {MODBUS_HOST}:{MODBUS_PORT}")
            else:
                raise ConnectionError("Modbus connect() returned
False")
        except Exception as e:
            log("ERROR", "p2l_controller", f"Modbus init failed:
{e}. Falling back to simulated.")
            self.protocol = "simulated"

    def _on_mqtt_connect(self, client, userdata, flags, rc):
        log("INFO", "p2l_controller", f"MQTT on_connect rc={rc}")
        self._connected = (rc == 0)

    def _on_mqtt_disconnect(self, client, userdata, rc):
        self._connected = False
        log("WARNING", "p2l_controller", f"MQTT disconnected
rc={rc}")

    # ── Signal Dispatch
──────────────────────────────────────────────────

    def _send_signal(self, bin_id: str, flash_count: int, color: str
= "GREEN") -> bool:
        payload = json.dumps({
            "bin_id":      bin_id,
            "flash_count": flash_count,
            "color":       color,
            "ts":          datetime.now().isoformat(),
```

```
        })

        if self.protocol == "mqtt" and self._connected:
            topic = f"{MQTT_TOPIC_BASE}/{bin_id}/flash"
            result = self._mqtt_client.publish(topic, payload)
            return result.rc == 0

        elif self.protocol == "modbus" and self._connected:
            # Map bin_id → Modbus register address (example mapping)
            reg = self._bin_to_modbus_register(bin_id)
            self._modbus_client.write_register(reg, flash_count)
            return True

        else:
            # Simulated: print signal, add tiny delay to mimic
hardware
            log("INFO", "p2l_controller",
                f"[SIM] BIN {bin_id} → {flash_count}x flash
({color}) | payload: {payload}")
            time.sleep(0.05)
            return True

    def _bin_to_modbus_register(self, bin_id: str) -> int:
        """Map BIN-XX → Modbus holding register index."""
        mapping = {
            "BIN-A1": 100, "BIN-A2": 101,
            "BIN-B1": 102, "BIN-B2": 103,
            "BIN-C1": 104, "BIN-C2": 105,
            "BIN-D1": 106, "BIN-D2": 107,
        }
        return mapping.get(bin_id, 200)

    # ── Public API
─────────────────────────────────────────────────────────────

    def signal_manifest(self, manifest: dict) -> list[dict]:
        """
        For every item in the manifest, flash the corresponding bin
light
        exactly `qty` times. Returns list of P2L event records.
```

```
        """
        order_id = manifest["order_id"]
        events   = []
        p2l_status.reset()

        all_items = manifest.get("meal_items", []) +
manifest.get("packaging", [])
        p2l_status.flash_queue = [i.get("name", "?") for i in
all_items]

        for item in all_items:
            name   = item.get("name", "Unknown")
            qty    = item.get("qty", 1)
            bin_id = item.get("bin_id", "BIN-UNKNOWN")

            if bin_id == "BIN-UNKNOWN":
                log("WARNING", "p2l_controller",
                    f"No bin mapping for item '{name}' in order
{order_id}")
                continue

            p2l_status.set_active(bin_id, name)

            success = self._send_signal(bin_id, qty)
            event = {
                "order_id":    order_id,
                "bin_id":      bin_id,
                "item_name":   name,
                "flash_count": qty,
                "success":     success,
                "triggered_at": datetime.now().isoformat(),
            }
            events.append(event)
            p2l_status.add_completed(event)
            log_p2l_event(order_id, bin_id, name, qty)

            if not success:
                log("ERROR", "p2l_controller",
                    f"Signal failed for {bin_id} ({name} x{qty})")
```

```
                p2l_status.error_msg = f"Signal error: {bin_id} —
{name}"

            time.sleep(0.1)  # inter-signal delay

        p2l_status.set_idle()
        log("INFO", "p2l_controller",
            f"P2L sequence complete for {order_id}: {len(events)}
bins signaled.")
        return events

    def flash_alert(self, bin_id: str = "BIN-ALL", color: str =
"RED"):
        """Send a red alert flash to all bins (weight error,
etc.)."""
        self._send_signal(bin_id, flash_count=5, color=color)
        log("ERROR", "p2l_controller", f"RED ALERT flashed on
{bin_id}")

    def is_connected(self) -> bool:
        if self.protocol == "simulated":
            return True
        return self._connected

    def get_protocol_label(self) -> str:
        labels = {
            "mqtt":      "MQTT (Live)",
            "modbus":    "Modbus TCP (Live)",
            "simulated": "Simulated",
        }
        return labels.get(self.protocol, self.protocol)

    def disconnect(self):
        if self.protocol == "mqtt" and self._mqtt_client:
            self._mqtt_client.loop_stop()
            self._mqtt_client.disconnect()
        elif self.protocol == "modbus" and self._modbus_client:
            self._modbus_client.close()
```

e. weather_engine.py

```
import os
import json
import requests
from datetime import datetime, date, timedelta
from pathlib import Path
from typing import Optional
from database import log

MOCK_DATA_PATH   = Path(__file__).parent / "mock_data.json"
OWM_API_KEY      = os.getenv("OWM_API_KEY", "")
WEATHER_BASE_URL = "https://api.openweathermap.org/data/2.5"

# ── FDA / USDA Thresholds ──────────────────────────────────────
EXTREME_HEAT_F    = 85.0
HOT_F             = 70.0
WARM_F            = 55.0
DANGER_ZONE_MAX_F = 40.0
FREEZE_RISK_F     = 32.0

# UI alias exports
HEAT_THRESHOLD_F   = HOT_F
FREEZE_THRESHOLD_F = FREEZE_RISK_F
PRECIP_THRESHOLD   = 50


def _load_mock() -> dict:
    with open(MOCK_DATA_PATH) as f:
        return json.load(f)


def _kelvin_to_f(k: float) -> float:
    return (k - 273.15) * 9 / 5 + 32


def get_safety_tier(temp_f: float) -> dict:
    if temp_f >= EXTREME_HEAT_F:
        return {
            "tier":     "EXTREME HEAT",
            "color":    "heat",
```

```
            "ice_qty":  3,
            "fda_note": f"≥{EXTREME_HEAT_F:.0f}°F — Interior exceeds
40°F (FDA danger zone) in under 1 hour. 3× Nutri-Ice required.",
        }
    if temp_f >= HOT_F:
        return {
            "tier":     "HOT",
            "color":    "heat",
            "ice_qty":  2,
            "fda_note": f"≥{HOT_F:.0f}°F — Interior exceeds 40°F in
~2 hours. 2× Nutri-Ice required.",
        }
    if temp_f >= WARM_F:
        return {
            "tier":     "WARM",
            "color":    "warn",
            "ice_qty":  2,
            "fda_note": f"≥{WARM_F:.0f}°F — Inside FDA danger zone
(40–140°F). Interior exceeds 40°F in ~4 hours. 2× Nutri-Ice
required.",
        }
    if temp_f >= DANGER_ZONE_MAX_F:
        return {
            "tier":     "MILD — ICE REQUIRED",
            "color":    "warn",
            "ice_qty":  1,
            "fda_note": f"≥{DANGER_ZONE_MAX_F:.0f}°F — At/above FDA
40°F threshold. 1× Nutri-Ice mandatory for all perishables.",
        }
    if temp_f >= FREEZE_RISK_F:
        return {
            "tier":     "COLD — SAFE",
            "color":    "safe",
            "ice_qty":  0,
            "fda_note":
f"{FREEZE_RISK_F:.0f}–{DANGER_ZONE_MAX_F:.0f}°F — Below FDA danger
zone. ClimaCell liner provides passive insulation. No ice needed.",
        }
    return {
        "tier":     "FREEZE RISK",
```

```
        "color":    "freeze",
        "ice_qty":  0,
        "fda_note": f"< {FREEZE_RISK_F:.0f}°F — Freeze damage risk.
ClimaCell liner insulates against freezing. No ice added.",
    }


class WeatherEngine:

    def __init__(self, use_mock: bool = True):
        self.use_mock = use_mock or not OWM_API_KEY
        self._mock    = _load_mock()
        self._catalog = {k: v for k, v in
self._mock.get("packing_materials", {}).items()
                         if not k.startswith("_")}

    def _fetch_live_current(self, zip_code: str) -> Optional[dict]:
        try:
            resp = requests.get(
                f"{WEATHER_BASE_URL}/weather",
                params={"zip": f"{zip_code},us", "appid":
OWM_API_KEY},
                timeout=8,
            )
            resp.raise_for_status()
            d = resp.json()
            return {
                "city":            d["name"],
                "current_temp_f":
round(_kelvin_to_f(d["main"]["temp"]), 1),
                "feels_like_f":
round(_kelvin_to_f(d["main"]["feels_like"]), 1),
                "condition":       d["weather"][0]["main"],
                "rain_chance_pct": int(d.get("pop", 0) * 100),
            }
        except Exception as e:
            log("WARNING", "weather_engine", f"OWM current fetch
failed for {zip_code}: {e}")
            return None
```

```
    def _fetch_live_forecast(self, zip_code: str, days: int = 5) ->
list[dict]:
        try:
            resp = requests.get(
                f"{WEATHER_BASE_URL}/forecast",
                params={"zip": f"{zip_code},us", "appid":
OWM_API_KEY, "cnt": days * 8},
                timeout=8,
            )
            resp.raise_for_status()
            raw   = resp.json().get("list", [])
            daily: dict[str, dict] = {}
            for entry in raw:
                day_str = entry["dt_txt"][:10]
                temp_f  = _kelvin_to_f(entry["main"]["temp_max"])
                rain    = int(entry.get("pop", 0) * 100)
                cond    = entry["weather"][0]["main"]
                if day_str not in daily:
                    daily[day_str] = {"date": day_str, "high_f":
temp_f,
                                      "low_f": temp_f,
"rain_chance_pct": rain, "condition": cond}
                else:
                    daily[day_str]["high_f"] =
max(daily[day_str]["high_f"], temp_f)
                    daily[day_str]["rain_chance_pct"] =
max(daily[day_str]["rain_chance_pct"], rain)
            return list(daily.values())[:days]
        except Exception as e:
            log("WARNING", "weather_engine", f"OWM forecast fetch
failed for {zip_code}: {e}")
            return []

    def get_weather(self, zip_code: str, delivery_date: str = "",
days_ahead: int = 5) -> dict:
        if self.use_mock:
            data = self._mock.get("mock_weather", {}).get(zip_code)
            if data:
                log("INFO", "weather_engine",
```

```
                    f"Mock weather for {zip_code}: {data['city']}
{data['current_temp_f']}°F")
                return data
            log("WARNING", "weather_engine",
                f"No mock data for {zip_code}. Using safe
conservative fallback (72°F → 2× Nutri-Ice).")
            return self._safe_fallback(zip_code)

        current = self._fetch_live_current(zip_code)
        if not current:
            log("WARNING", "weather_engine",
                f"Weather API unavailable for {zip_code}.
Conservative fallback applied.")
            return self._safe_fallback(zip_code)
        current["forecast"] = self._fetch_live_forecast(zip_code,
days=days_ahead)
        return current

    def _safe_fallback(self, zip_code: str) -> dict:
        """
        Conservative fallback: 72°F triggers the WARM tier → 2×
Nutri-Ice.
        Never under-protect perishables when weather data is
unavailable.
        """
        today = date.today()
        return {
            "city":            f"ZIP {zip_code}",
            "current_temp_f":  72.0,
            "feels_like_f":    72.0,
            "condition":       "Unknown",
            "rain_chance_pct": 0,
            "is_fallback":     True,
            "forecast": [
                {"date": str(today + timedelta(days=i)), "high_f":
75, "low_f": 60,
                 "rain_chance_pct": 20, "condition": "Unknown"}
                for i in range(5)
            ],
        }
```

```
    def apply_weather_rules(self, base_packing: list[dict],
                            weather: dict, order: dict) ->
list[dict]:
        """
        Adjust Nutri-Ice quantity based on FDA temperature tier.

        THIS ENGINE ONLY TOUCHES THE ICE PACK.
        All other packaging (box, ClimaCell liner, paper kit bag,
recipe card,
        autobagger bags) is already set by api_client.py and is left
unchanged.
        """
        packing  = {item["name"]: item.copy() for item in
base_packing}
        temp     = weather["current_temp_f"]
        rain_pct = weather["rain_chance_pct"]

        # Worst-case temperature and precipitation across the full
delivery window
        forecast_max_temp = max(
            (f.get("high_f", temp) for f in weather.get("forecast",
[])), default=temp
        )
        forecast_max_rain = max(
            (f.get("rain_chance_pct", 0) for f in
weather.get("forecast", [])), default=rain_pct
        )
        effective_temp = max(temp, forecast_max_temp)
        effective_rain = max(rain_pct, forecast_max_rain)

        tier = get_safety_tier(effective_temp)
        log("INFO", "weather_engine",
            f"Order {order['order_id']} | {effective_temp:.0f}°F |
tier: {tier['tier']} | "
            f"Nutri-Ice qty: {tier['ice_qty']} | rain:
{effective_rain}%")

        # ── Nutri-Ice: set qty from FDA tier
_____________________________________
```

```
        # Remove the baseline pack first; re-add with correct qty
        packing.pop("Nutri-Ice Pack", None)

        if tier["ice_qty"] > 0:
            nutrice = self._catalog.get("Nutri-Ice Pack", {})
            packing["Nutri-Ice Pack"] = {
                **nutrice,
                "name":     "Nutri-Ice Pack",
                "qty":      tier["ice_qty"],
                "source":   "tier1-fda",
                "tier":     "tier1",
                "fda_rule": f"FDA §3-501.11: {tier['tier']} —
{tier['ice_qty']}× Nutri-Ice required",
                "note":     tier["fda_note"],
            }
            log("INFO", "weather_engine",
                f"Nutri-Ice: {tier['ice_qty']}× applied
({tier['tier']})")
        else:
            log("INFO", "weather_engine",
                f"Nutri-Ice: 0× — {tier['tier']}
({effective_temp:.0f}°F). "
                "ClimaCell liner provides passive cold protection.")

        return list(packing.values())

    def build_manifest(self, order: dict, base_packing: list[dict],
                       weather: dict) -> dict:
        adjusted_packing = self.apply_weather_rules(base_packing,
weather, order)

        items_weight   = sum(i["weight_kg"] * i.get("qty", 1) for i
in order["items"])
        pkg_weight     = sum(p.get("weight_kg", 0) * p.get("qty", 1)
for p in adjusted_packing)
        theoretical_kg = round(items_weight + pkg_weight, 3)

        temp = weather["current_temp_f"]
        eff  = max(temp, max((f.get("high_f", temp) for f in
weather.get("forecast", [])), default=temp))
```

```
        tier = get_safety_tier(eff)

        cond = weather["condition"].lower()
        if temp < FREEZE_RISK_F or "snow" in cond or "sleet" in
cond:
            badge = "❄️ FREEZE RISK"
        elif "thunder" in cond or "storm" in cond:
            badge = "⛈️ STORM"
        elif "rain" in cond or "drizzle" in cond:
            badge = "🌧️ RAIN"
        elif temp >= EXTREME_HEAT_F:
            badge = "🌡️ EXTREME HEAT"
        elif temp >= HOT_F:
            badge = "☀️ HOT"
        elif temp >= WARM_F:
            badge = "🌤️ WARM"
        elif temp >= DANGER_ZONE_MAX_F:
            badge = "🌥️ MILD"
        else:
            badge = "🥶 COLD — SAFE"

        return {
            "manifest_version": "3.1",
            "generated_at":     datetime.now().isoformat(),
            "order_id":         order["order_id"],
            "customer":         order["customer_name"],
            "destination": {
                "address": order["delivery_address"],
                "city":    order["city"],
                "state":   order["state"],
                "zip":     order["zip_code"],
            },
            "delivery_date": order.get("delivery_date", ""),
            "food_safety": {
                "standard":  "FDA Food Code 2022 §3-501.11 + USDA
FSIS Danger Zone",
                "danger_zone": "40°F – 140°F",
                "tier":      tier["tier"],
                "fda_note":  tier["fda_note"],
                "ice_qty":   tier["ice_qty"],
```

```
                "ice_brand": "Nutri-Ice (Tier 1)",
                "compliant": True,
            },
            "weather_summary": {
                "city":            weather.get("city", ""),
                "current_temp_f":  temp,
                "feels_like_f":    weather.get("feels_like_f",
temp),
                "condition":       weather["condition"],
                "rain_chance_pct": weather["rain_chance_pct"],
                "badge":           badge,
                "is_fallback":     weather.get("is_fallback",
False),
                "forecast":        weather.get("forecast", []),
            },
            "sustainability": {
                "tier1_applied": "Nutri-Ice (replaces Pelton
Shepherd)",
                "tier2_applied": "Paper autobagger bags (replaces
clear perforated plastic)",
                "tier2_bags_replaced": [
                    p["name"] for p in adjusted_packing if
p.get("tier") == "tier2"
                ],
                "not_modelled": [
                    "Sauce containers", "Meat packaging", "Spice
sachets",
                    "Kit label"
                ],
            },
            "meal_items": order["items"],
            "packaging":  adjusted_packing,
            "weights": {
                "meal_items_kg":        round(items_weight, 3),
                "packaging_kg":         round(pkg_weight, 3),
                "theoretical_total_kg": theoretical_kg,
                "actual_kg":            None,
                "weight_ok":            None,
            },
        }
```

```
    def get_weather_icon(self, weather: dict) -> str:
        temp = weather.get("current_temp_f", 68)
        cond = weather.get("condition", "").lower()
        rain = weather.get("rain_chance_pct", 0)
        if "snow" in cond or "sleet" in cond or temp <
FREEZE_RISK_F: return "❄️"
        if "thunder" in cond or "storm" in cond:
return "⛈️"
        if "rain" in cond or "drizzle" in cond or rain >
PRECIP_THRESHOLD: return "🌧️"
        if temp >= EXTREME_HEAT_F:  return "🌡️"
        if temp >= HOT_F:           return "☀️"
        if temp >= WARM_F:          return "🌤️"
        return "⛅"
```

f. requirements.txt

```
streamlit>=1.35.0
requests>=2.31.0
paho-mqtt>=2.0.0
pymodbus>=3.6.0
```

g. mock_data.json

```
{
  "katana_orders": [
    {
      "order_id": "GC-2026-0501",
      "customer_name": "Sarah Mitchell",
      "delivery_date": "2026-05-06",
      "delivery_address": "4521 Magnolia Blvd",
      "city": "Phoenix",
      "state": "AZ",
      "zip_code": "85001",
      "items": [
        {"sku": "GC-ING-001", "name": "Boneless Chicken Thighs",
"qty": 2, "weight_kg": 0.340, "unit": "pieces", "category":
"protein",  "bag_type": "autobagger_8x13"},
```

```
        {"sku": "GC-ING-002", "name": "Lemon",
"qty": 2, "weight_kg": 0.100, "unit": "pieces", "category":
"produce",  "bag_type": "autobagger_5x11_a"},
        {"sku": "GC-ING-003", "name": "Fresh Thyme",
"qty": 1, "weight_kg": 0.020, "unit": "sprig",  "category": "herb",
"bag_type": "autobagger_5x11_a"},
        {"sku": "GC-ING-004", "name": "Garlic Cloves",
"qty": 4, "weight_kg": 0.016, "unit": "pieces", "category":
"produce",  "bag_type": "autobagger_5x11_a"},
        {"sku": "GC-ING-005", "name": "Baby Spinach",
"qty": 1, "weight_kg": 0.142, "unit": "bag",    "category":
"produce",  "bag_type": "autobagger_5x11_b"},
        {"sku": "GC-ING-006", "name": "Cherry Tomatoes",
"qty": 1, "weight_kg": 0.227, "unit": "punnet", "category":
"produce",  "bag_type": "autobagger_5x11_b"},
        {"sku": "GC-ING-007", "name": "Olive Oil Portion",
"qty": 1, "weight_kg": 0.030, "unit": "sachet", "category":
"pantry",   "bag_type": "none"},
        {"sku": "GC-ING-008", "name": "Herb Seasoning Blend",
"qty": 1, "weight_kg": 0.015, "unit": "sachet", "category":
"pantry",   "bag_type": "none"},
        {"sku": "GC-ING-009", "name": "Balsamic Glaze",
"qty": 1, "weight_kg": 0.040, "unit": "sachet", "category": "sauce",
"bag_type": "none"},
        {"sku": "GC-ING-010", "name": "Orzo Pasta",
"qty": 1, "weight_kg": 0.200, "unit": "bag",    "category": "grain",
"bag_type": "autobagger_8x13"}
      ],
      "status": "pending"
    },
    {
      "order_id": "GC-2026-0502",
      "customer_name": "James Okonkwo",
      "delivery_date": "2026-05-08",
      "delivery_address": "889 Birchwood Ave",
      "city": "Minneapolis",
      "state": "MN",
      "zip_code": "55401",
      "items": [
```

```
        {"sku": "GC-ING-011", "name": "Grass-Fed Ground Beef 8oz",
"qty": 2, "weight_kg": 0.454, "unit": "pack",   "category":
"protein",  "bag_type": "autobagger_8x13"},
        {"sku": "GC-ING-012", "name": "Russet Potatoes",
"qty": 3, "weight_kg": 0.200, "unit": "pieces", "category":
"produce",  "bag_type": "autobagger_8x13"},
        {"sku": "GC-ING-013", "name": "Cremini Mushrooms",
"qty": 1, "weight_kg": 0.227, "unit": "pack",   "category":
"produce",  "bag_type": "autobagger_5x11_b"},
        {"sku": "GC-ING-014", "name": "Yellow Onion",
"qty": 1, "weight_kg": 0.180, "unit": "pieces", "category":
"produce",  "bag_type": "autobagger_5x11_b"},
        {"sku": "GC-ING-015", "name": "Beef Bone Broth Portion",
"qty": 1, "weight_kg": 0.120, "unit": "carton", "category":
"pantry",   "bag_type": "none"},
        {"sku": "GC-ING-016", "name": "Sour Cream Portion",
"qty": 1, "weight_kg": 0.057, "unit": "cup",    "category": "dairy",
"bag_type": "none"},
        {"sku": "GC-ING-017", "name": "Fresh Chives",
"qty": 1, "weight_kg": 0.014, "unit": "bunch",  "category": "herb",
"bag_type": "autobagger_5x11_a"},
        {"sku": "GC-ING-018", "name": "Smoked Paprika Blend",
"qty": 1, "weight_kg": 0.012, "unit": "sachet", "category":
"pantry",   "bag_type": "none"},
        {"sku": "GC-ING-019", "name": "Shredded Cheddar Cheese",
"qty": 1, "weight_kg": 0.113, "unit": "bag",    "category": "dairy",
"bag_type": "autobagger_5x11_b"},
        {"sku": "GC-ING-020", "name": "Unsalted Butter Portion",
"qty": 2, "weight_kg": 0.028, "unit": "pack",   "category": "dairy",
"bag_type": "none"}
      ],
      "status": "pending"
    },
    {
      "order_id": "GC-2026-0503",
      "customer_name": "Elena Vasquez",
      "delivery_date": "2026-05-10",
      "delivery_address": "230 Ocean Drive",
      "city": "Miami",
      "state": "FL",
```

```
      "zip_code": "33101",
      "items": [
        {"sku": "GC-ING-021", "name": "Wild-Caught Shrimp Peeled",
"qty": 2, "weight_kg": 0.340, "unit": "bag",    "category":
"protein",  "bag_type": "autobagger_8x13"},
        {"sku": "GC-ING-022", "name": "Ripe Avocados",
"qty": 2, "weight_kg": 0.170, "unit": "pieces", "category":
"produce",  "bag_type": "autobagger_5x11_b"},
        {"sku": "GC-ING-023", "name": "Corn Tortillas 6-pack",
"qty": 1, "weight_kg": 0.198, "unit": "pack",   "category": "grain",
"bag_type": "autobagger_8x13"},
        {"sku": "GC-ING-024", "name": "Shredded Red Cabbage",
"qty": 1, "weight_kg": 0.142, "unit": "bag",    "category":
"produce",  "bag_type": "autobagger_5x11_b"},
        {"sku": "GC-ING-025", "name": "Lime",
"qty": 2, "weight_kg": 0.067, "unit": "pieces", "category":
"produce",  "bag_type": "autobagger_5x11_a"},
        {"sku": "GC-ING-026", "name": "Jalapeno",
"qty": 1, "weight_kg": 0.050, "unit": "pieces", "category":
"produce",  "bag_type": "autobagger_5x11_a"},
        {"sku": "GC-ING-027", "name": "Fresh Cilantro",
"qty": 1, "weight_kg": 0.028, "unit": "bunch",  "category": "herb",
"bag_type": "autobagger_5x11_a"},
        {"sku": "GC-ING-028", "name": "Chipotle Crema",
"qty": 1, "weight_kg": 0.057, "unit": "sachet", "category": "sauce",
"bag_type": "none"},
        {"sku": "GC-ING-029", "name": "Cumin and Chili Spice Blend",
"qty": 1, "weight_kg": 0.012, "unit": "sachet", "category":
"pantry",   "bag_type": "none"},
        {"sku": "GC-ING-030", "name": "Crumbled Cotija Cheese",
"qty": 1, "weight_kg": 0.057, "unit": "bag",    "category": "dairy",
"bag_type": "autobagger_5x11_a"}
      ],
      "status": "pending"
    },
    {
      "order_id": "GC-2026-0504",
      "customer_name": "Daniel Park",
      "delivery_date": "2026-05-13",
      "delivery_address": "17 Beacon Hill Rd",
```

```
      "city": "Boston",
      "state": "MA",
      "zip_code": "02108",
      "items": [
        {"sku": "GC-ING-031", "name": "Atlantic Salmon Fillet 6oz",
"qty": 2, "weight_kg": 0.340, "unit": "fillet", "category":
"protein",  "bag_type": "autobagger_8x13"},
        {"sku": "GC-ING-032", "name": "Asparagus Spears",
"qty": 1, "weight_kg": 0.227, "unit": "bunch",  "category":
"produce",  "bag_type": "autobagger_8x13"},
        {"sku": "GC-ING-033", "name": "Baby Yukon Gold Potatoes",
"qty": 1, "weight_kg": 0.340, "unit": "bag",    "category":
"produce",  "bag_type": "autobagger_8x13"},
        {"sku": "GC-ING-034", "name": "Capers",
"qty": 1, "weight_kg": 0.030, "unit": "jar",    "category":
"pantry",   "bag_type": "none"},
        {"sku": "GC-ING-035", "name": "Fresh Dill",
"qty": 1, "weight_kg": 0.014, "unit": "bunch",  "category": "herb",
"bag_type": "autobagger_5x11_a"},
        {"sku": "GC-ING-036", "name": "Creme Fraiche Portion",
"qty": 1, "weight_kg": 0.057, "unit": "cup",    "category": "dairy",
"bag_type": "none"},
        {"sku": "GC-ING-037", "name": "Dijon Mustard Portion",
"qty": 1, "weight_kg": 0.021, "unit": "sachet", "category":
"pantry",   "bag_type": "none"},
        {"sku": "GC-ING-038", "name": "Lemon Caper Butter Blend",
"qty": 1, "weight_kg": 0.035, "unit": "sachet", "category": "sauce",
"bag_type": "none"},
        {"sku": "GC-ING-039", "name": "Olive Oil Portion",
"qty": 1, "weight_kg": 0.030, "unit": "sachet", "category":
"pantry",   "bag_type": "none"},
        {"sku": "GC-ING-040", "name": "Sea Salt and Cracked Pepper",
"qty": 1, "weight_kg": 0.010, "unit": "sachet", "category":
"pantry",   "bag_type": "none"}
      ],
      "status": "pending"
    },
    {
      "order_id": "GC-2026-0505",
      "customer_name": "Priya Sharma",
```

```
      "delivery_date": "2026-05-16",
      "delivery_address": "602 Wacker Dr Apt 14F",
      "city": "Chicago",
      "state": "IL",
      "zip_code": "60601",
      "items": [
        {"sku": "GC-ING-041", "name": "Firm Paneer Cubes",
"qty": 1, "weight_kg": 0.250, "unit": "pack",   "category":
"protein",  "bag_type": "autobagger_8x13"},
        {"sku": "GC-ING-042", "name": "Basmati Rice",
"qty": 1, "weight_kg": 0.200, "unit": "bag",    "category": "grain",
"bag_type": "autobagger_8x13"},
        {"sku": "GC-ING-043", "name": "Diced Tomatoes Canned",
"qty": 1, "weight_kg": 0.227, "unit": "can",    "category":
"pantry",   "bag_type": "none"},
        {"sku": "GC-ING-044", "name": "Coconut Milk Portion",
"qty": 1, "weight_kg": 0.180, "unit": "carton", "category":
"pantry",   "bag_type": "none"},
        {"sku": "GC-ING-045", "name": "Fresh Ginger Knob",
"qty": 1, "weight_kg": 0.030, "unit": "pieces", "category":
"produce",  "bag_type": "autobagger_5x11_a"},
        {"sku": "GC-ING-046", "name": "Garlic Cloves",
"qty": 3, "weight_kg": 0.016, "unit": "pieces", "category":
"produce",  "bag_type": "autobagger_5x11_a"},
        {"sku": "GC-ING-047", "name": "Frozen Baby Peas Portion",
"qty": 1, "weight_kg": 0.113, "unit": "bag",    "category":
"produce",  "bag_type": "autobagger_5x11_b"},
        {"sku": "GC-ING-048", "name": "Tikka Masala Spice Blend",
"qty": 1, "weight_kg": 0.018, "unit": "sachet", "category":
"pantry",   "bag_type": "none"},
        {"sku": "GC-ING-049", "name": "Naan Bread 2-pack",
"qty": 1, "weight_kg": 0.170, "unit": "pack",   "category": "grain",
"bag_type": "autobagger_8x13"},
        {"sku": "GC-ING-050", "name": "Greek Yogurt Portion",
"qty": 1, "weight_kg": 0.113, "unit": "cup",    "category": "dairy",
"bag_type": "none"}
      ],
      "status": "pending"
    }
  ],
```

```
  "_bom_notes": {
    "tier1_scope": "Ice pack only: Pelton Shepherd → Nutri-Ice. All
other box components unchanged.",
    "tier2_scope": "Plastic autobagger bags only. Three variants:
autobagger_5x11_a, autobagger_5x11_b, autobagger_8x13.",
    "excluded_from_tier2": [
      "Exterior cardboard box — already sustainable",
      "ClimaCell liner — already plant-based and curbside
recyclable",
      "Paper kit bag — already paper",
      "Recipe card — already paper",
      "Kit label — not modelled",
      "Sauce containers — no cost data",
      "Meat packaging — no cost data",
      "Spice sachets — not broken out in cost file"
    ]
  },

  "packing_materials": {
    "_comment_box":    "Exterior corrugated cardboard box — already
sustainable, no change",
    "Standard Box":    {"sku": "PKG-BOX-STD",  "bin_id": "BIN-D1",
"weight_kg": 0.85,  "sustainable": true,  "tier": "base",
"description": "Exterior corrugated cardboard box — already
sustainable"},

    "_comment_liner":  "ClimaCell liner — plant-based EPS
alternative, curbside recyclable, no change",
    "ClimaCell Liner": {"sku": "PKG-CLIMACELL","bin_id": "BIN-B1",
"weight_kg": 0.22,  "sustainable": true,  "tier": "base",
"description": "Plant-based insulated liner, curbside recyclable —
no change needed"},

    "_comment_kitbag": "Paper kit bag — already paper, no change",
    "Paper Kit Bag":   {"sku": "PKG-KITBAG",   "bin_id": "BIN-E1",
"weight_kg": 0.04,  "sustainable": true,  "tier": "base",
"description": "Paper-based kit bag holding all ingredients —
already sustainable"},
```

```
    "_comment_card":   "Recipe card — already paper, no change",
    "Recipe Card":     {"sku": "PKG-CARD",     "bin_id": "BIN-E2",
"weight_kg": 0.015, "sustainable": true,  "tier": "base",
"description": "Paper recipe card — already sustainable"},

    "_comment_ice_t1": "TIER 1: Nutri-Ice replaces Pelton Shepherd
standard ice pack. Scope = ice pack only.",
    "Nutri-Ice Pack":  {"sku": "PKG-NUTRICE",  "bin_id": "BIN-A1",
"weight_kg": 0.340, "sustainable": true,  "tier": "tier1",
"description": "Nutri-Ice — Tier 1 replacement for Pelton Shepherd
standard ice pack"},

    "_comment_bags":   "TIER 2: Autobagger plastic bags →
paper/sustainable alternative (cost modelled only for these three
variants)",
    "Autobagger 5x11 Small (Paper Alt)": {"sku": "PKG-BAG-5S",
"bin_id": "BIN-F1", "weight_kg": 0.008, "sustainable": true,
"tier": "tier2", "description": "Replaces Autobagger 5×11.5 small
clear perforated plastic bag"},
    "Autobagger 5x11 Alt (Paper Alt)":   {"sku": "PKG-BAG-5A",
"bin_id": "BIN-F2", "weight_kg": 0.009, "sustainable": true,
"tier": "tier2", "description": "Replaces Autobagger 5×11.5 alt
clear perforated plastic bag"},
    "Autobagger 8x13 (Paper Alt)":       {"sku": "PKG-BAG-8",
"bin_id": "BIN-F3", "weight_kg": 0.012, "sustainable": true,
"tier": "tier2", "description": "Replaces Autobagger 8×13.5 clear
perforated plastic bag"},

    "_comment_not_modelled": "The following use their ORIGINAL
packaging — no cost data or sustainable alternative available yet",
    "Sauce Container":     {"sku": "PKG-SAUCE", "bin_id": "BIN-G1",
"weight_kg": 0.020, "sustainable": false, "tier": "not-modelled",
"description": "Sauce container — no cost data for alternative"},
    "Meat Packaging":      {"sku": "PKG-MEAT",  "bin_id": "BIN-G2",
"weight_kg": 0.025, "sustainable": false, "tier": "not-modelled",
"description": "Meat/protein packaging — no cost data for
alternative"},
    "Spice Sachet":        {"sku": "PKG-SPICE", "bin_id": "BIN-G3",
"weight_kg": 0.005, "sustainable": false, "tier": "not-modelled",
"description": "Spice sachet — not broken out in cost file"}
```

```
  },

  "mock_weather": {
    "85001": {
      "city": "Phoenix",
      "current_temp_f": 97,
      "feels_like_f": 104,
      "condition": "Sunny",
      "rain_chance_pct": 4,
      "forecast": [
        {"date": "2026-05-06", "high_f": 101, "low_f": 80,
"rain_chance_pct": 3,  "condition": "Sunny"},
        {"date": "2026-05-07", "high_f": 104, "low_f": 83,
"rain_chance_pct": 5,  "condition": "Sunny"},
        {"date": "2026-05-08", "high_f": 99,  "low_f": 78,
"rain_chance_pct": 10, "condition": "Partly Cloudy"},
        {"date": "2026-05-09", "high_f": 96,  "low_f": 76,
"rain_chance_pct": 8,  "condition": "Sunny"},
        {"date": "2026-05-10", "high_f": 102, "low_f": 81,
"rain_chance_pct": 2,  "condition": "Sunny"}
      ]
    },
    "55401": {
      "city": "Minneapolis",
      "current_temp_f": 52,
      "feels_like_f": 47,
      "condition": "Partly Cloudy",
      "rain_chance_pct": 55,
      "forecast": [
        {"date": "2026-05-08", "high_f": 55, "low_f": 44,
"rain_chance_pct": 60, "condition": "Rain"},
        {"date": "2026-05-09", "high_f": 50, "low_f": 40,
"rain_chance_pct": 70, "condition": "Heavy Rain"},
        {"date": "2026-05-10", "high_f": 58, "low_f": 45,
"rain_chance_pct": 45, "condition": "Cloudy"},
        {"date": "2026-05-11", "high_f": 62, "low_f": 48,
"rain_chance_pct": 30, "condition": "Partly Cloudy"},
        {"date": "2026-05-12", "high_f": 65, "low_f": 50,
"rain_chance_pct": 20, "condition": "Mostly Sunny"}
      ]
```

```
    },
    "33101": {
      "city": "Miami",
      "current_temp_f": 88,
      "feels_like_f": 95,
      "condition": "Thunderstorm",
      "rain_chance_pct": 80,
      "forecast": [
        {"date": "2026-05-10", "high_f": 91, "low_f": 79,
"rain_chance_pct": 85, "condition": "Thunderstorm"},
        {"date": "2026-05-11", "high_f": 89, "low_f": 77,
"rain_chance_pct": 75, "condition": "Rain"},
        {"date": "2026-05-12", "high_f": 87, "low_f": 76,
"rain_chance_pct": 60, "condition": "Showers"},
        {"date": "2026-05-13", "high_f": 88, "low_f": 77,
"rain_chance_pct": 55, "condition": "Partly Cloudy"},
        {"date": "2026-05-14", "high_f": 90, "low_f": 78,
"rain_chance_pct": 40, "condition": "Partly Cloudy"}
      ]
    },
    "02108": {
      "city": "Boston",
      "current_temp_f": 61,
      "feels_like_f": 58,
      "condition": "Partly Cloudy",
      "rain_chance_pct": 35,
      "forecast": [
        {"date": "2026-05-13", "high_f": 63, "low_f": 52,
"rain_chance_pct": 40, "condition": "Showers"},
        {"date": "2026-05-14", "high_f": 67, "low_f": 54,
"rain_chance_pct": 25, "condition": "Partly Cloudy"},
        {"date": "2026-05-15", "high_f": 70, "low_f": 56,
"rain_chance_pct": 15, "condition": "Sunny"},
        {"date": "2026-05-16", "high_f": 72, "low_f": 58,
"rain_chance_pct": 10, "condition": "Sunny"},
        {"date": "2026-05-17", "high_f": 68, "low_f": 55,
"rain_chance_pct": 30, "condition": "Partly Cloudy"}
      ]
    },
    "60601": {
```

```
      "city": "Chicago",
      "current_temp_f": 65,
      "feels_like_f": 62,
      "condition": "Partly Cloudy",
      "rain_chance_pct": 30,
      "forecast": [
        {"date": "2026-05-16", "high_f": 68, "low_f": 55,
"rain_chance_pct": 35, "condition": "Showers"},
        {"date": "2026-05-17", "high_f": 72, "low_f": 58,
"rain_chance_pct": 20, "condition": "Partly Cloudy"},
        {"date": "2026-05-18", "high_f": 75, "low_f": 60,
"rain_chance_pct": 15, "condition": "Sunny"},
        {"date": "2026-05-19", "high_f": 73, "low_f": 59,
"rain_chance_pct": 25, "condition": "Partly Cloudy"},
        {"date": "2026-05-20", "high_f": 70, "low_f": 57,
"rain_chance_pct": 40, "condition": "Cloudy"}
      ]
    }
  }
}
```